



Chapter 04

기본 위젯 익히기

백제직업전문학교

김영준 강사



이번 장에서는 무엇을 다룰까요?



화면을 먼저 만들어보고 싶어요.



- 뷰란 무엇일까요?
- 레이아웃을 이용해 뷰를 화면에 배치해 볼까요?
- 텍스트뷰나 버튼과 같은 기본 위젯을 추가해 볼까요?



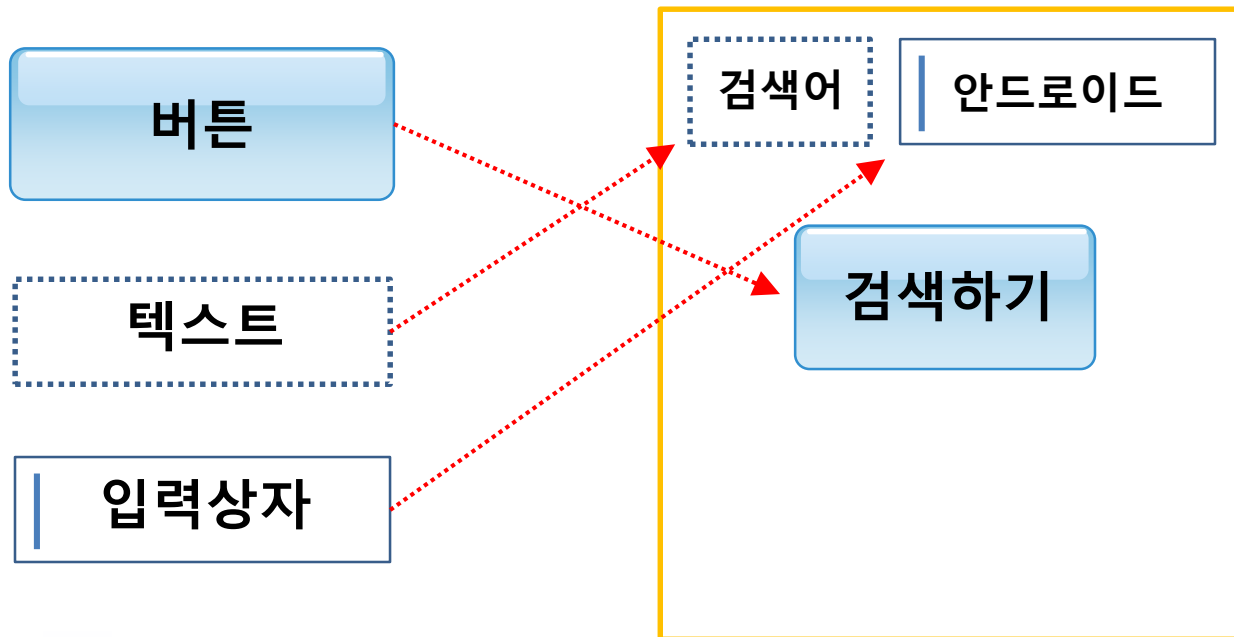


이번 장에서는 무엇을 다룰까요?

뷰란 무엇일까요?

레이아웃을 이용해
뷰를 화면에 배치해 볼까요?

기본 위젯 사용하기



텍스트

```
<TextView  
android:id="@+id/textview01"  
android:layout_width="wrap_content"  
android:layout_height="wrap_content"  
android:text="검색어"  
>
```



1. 뷰의 개요 ▶ 뷰와 뷰그룹

❖ View 클래스

- 안드로이드 화면에서 실제로 사용되는 것들은 모두 View 클래스 상속을 받음
- 다른 말로 '위젯' 이라고도 함
 - 화면에서 버튼 ➔ 버튼 위젯 / 실제 코드에서 버튼 ➔ 버튼 클래스
- 다른 위젯을 담을 수 있는 위젯은 레이아웃이라고 함
- 레이아웃은 ViewGroup 클래스 아래 존재
- 레이아웃도 크게 보면 위젯에 포함됨

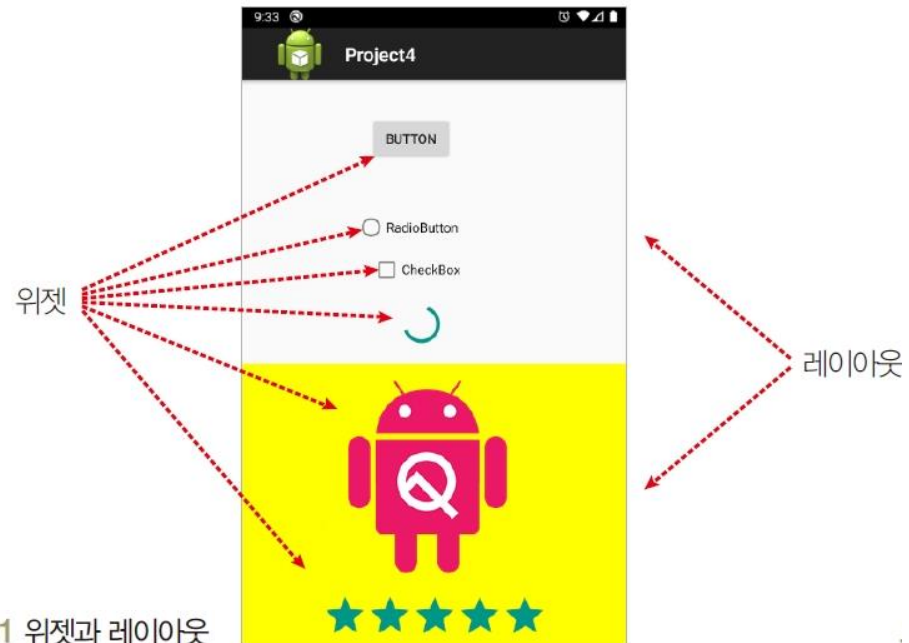


그림 4-1 위젯과 레이아웃

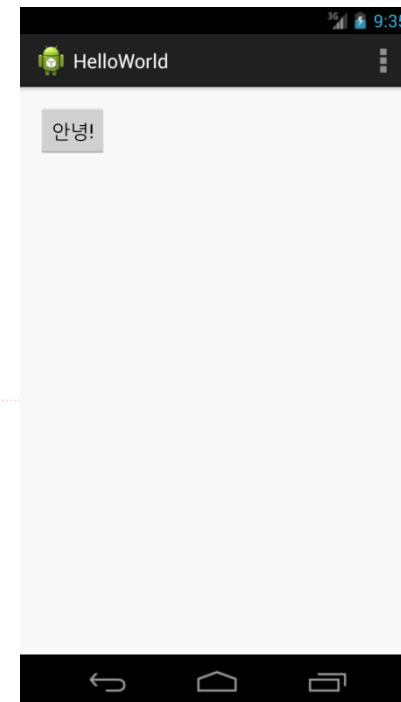


꺅쇱 안의 글자를 Button으로 바꾸기

- 태그의 글자가 TextView에서 Button으로 바뀌기만 해도 버튼이 됨

<Button

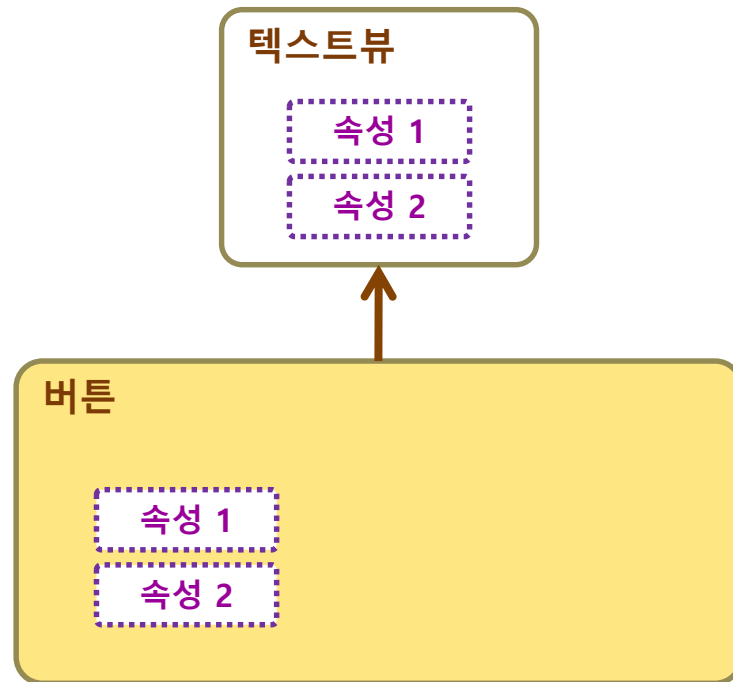
```
android:layout_width= "wrap_content"  
android:layout_height= "wrap_content"  
android:text= "@string/hello_world" />
```





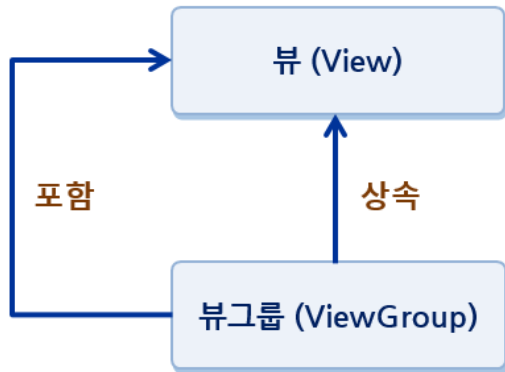
버튼의 속성과 텍스트뷰 속성의 관계

- 버튼은 텍스트뷰를 상속함 (상속이 어떤 의미를 가지고 있는지는 나중에 다시 설명함)
- 버튼이 텍스트뷰의 속성을 모두 가지고 있으면서 버튼만의 속성을 더 추가한 것

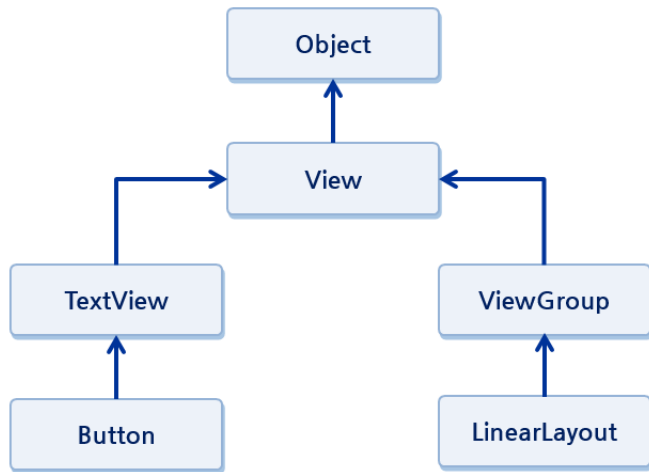




뷰와 뷰그룹의 정의



[뷰와 뷰 그룹의 관계]



[버튼과 리니어 레이아웃의 계층도]

• 뷰(View)

- 화면에 보이는 각각의 것들 (버튼, 텍스트 등등)
- 흔히 컨트롤(Control)이나 위젯(Widget)이라 불리는 UI 구성 요소

• 뷰 그룹(View Group)

- 뷰 들을 여러 개 포함하고 있는 것
- 뷰 그룹도 뷰에서 상속하여 뷰가 됨. 즉, 위의 뷰는 버튼, 텍스트 뿐만 아니라 이것들을 포함하는 눈에 보이지 않는 영역을 포함함

• 위젯(Widget)

- 뷰 중에서 일반적인 컨트롤의 역할을 하고 있는 것
- 버튼, 텍스트 등등

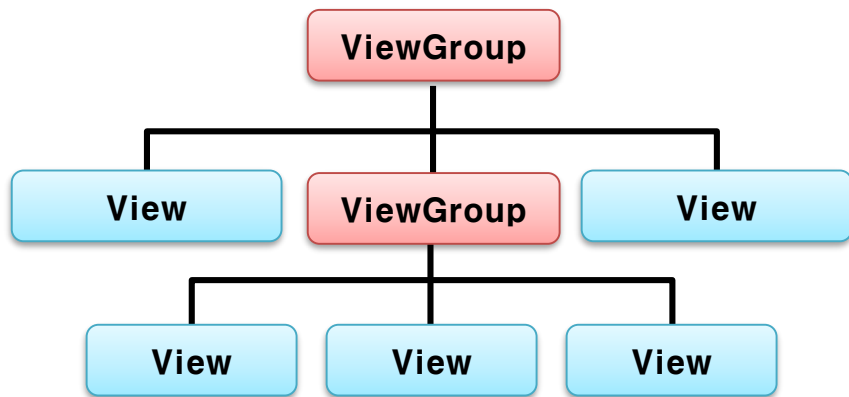
• 레이아웃(Layout)

- 뷰 그룹 중에서 내부에 뷰들을 포함하고 있으면서 그것들을 배치하는 역할을 하는 것

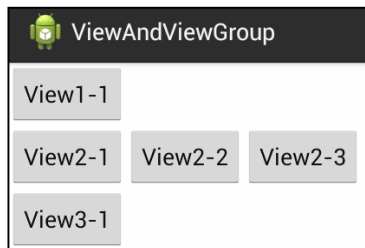


뷰와 뷰그룹의 정의

■ 왜 뷰그룹이 뷰를 상속받아야 할까?



뷰그룹 자체도 부모 뷰그룹에 의해 배치되어야 하기 때문이다.



● 레이아웃 구조

- LinearLayout
 - Button - "View1-1"
 - LinearLayout
 - Button - "View2-1"
 - Button - "View2-2"
 - Button - "View2-3"
 - Button - "View3-1"



1. 뷰의 개요 ▶ 뷰와 뷰그룹

❖ View 클래스 계층도

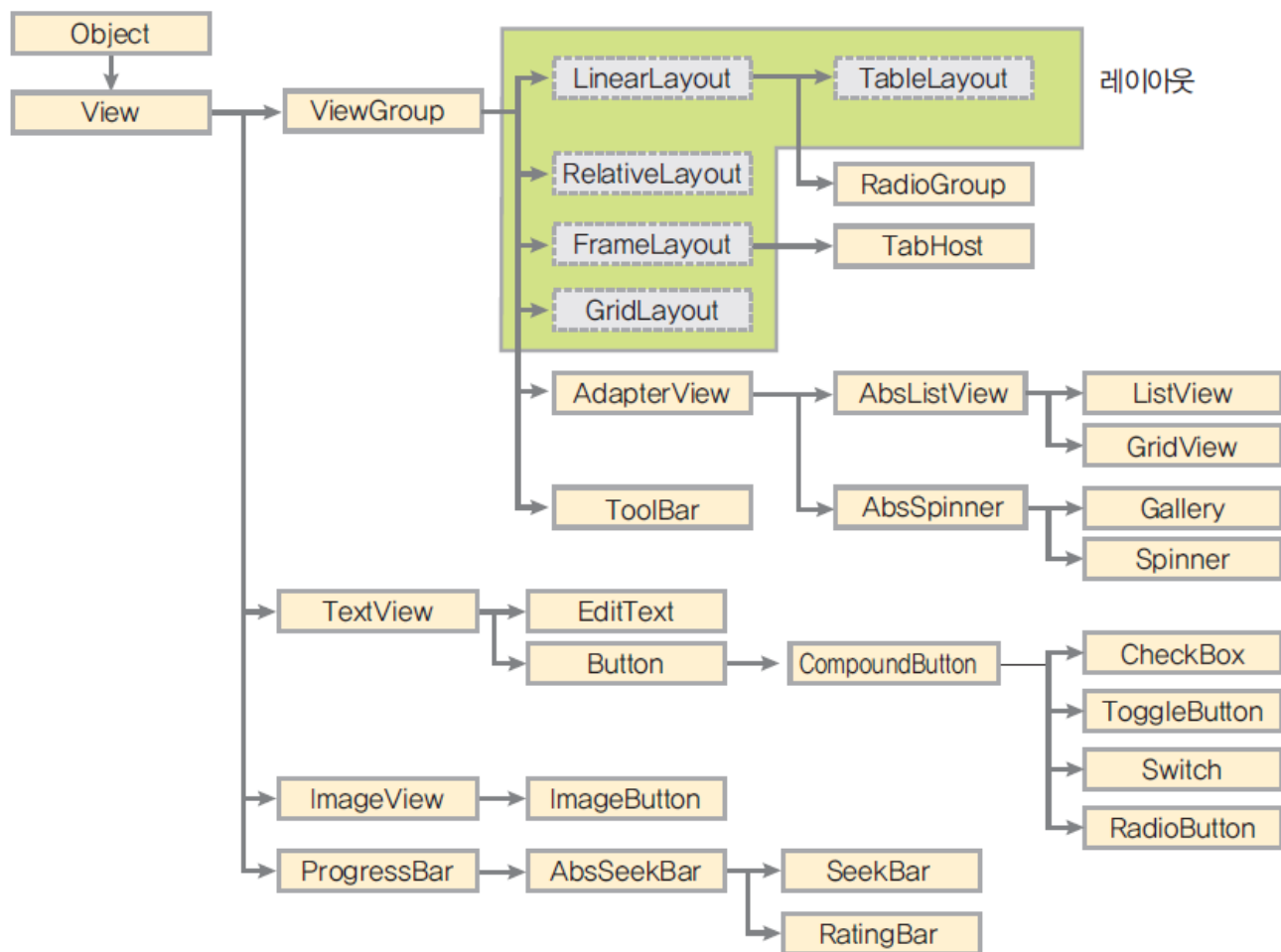
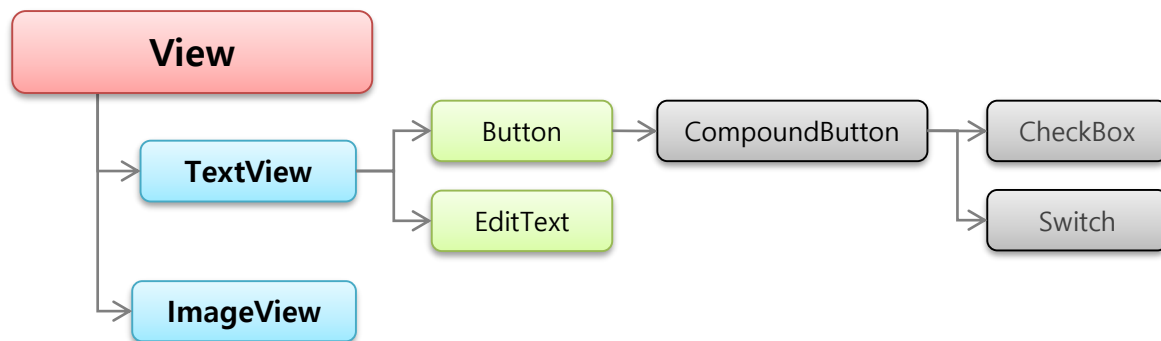


그림 4-2 안드로이드 View 클래스 계층도



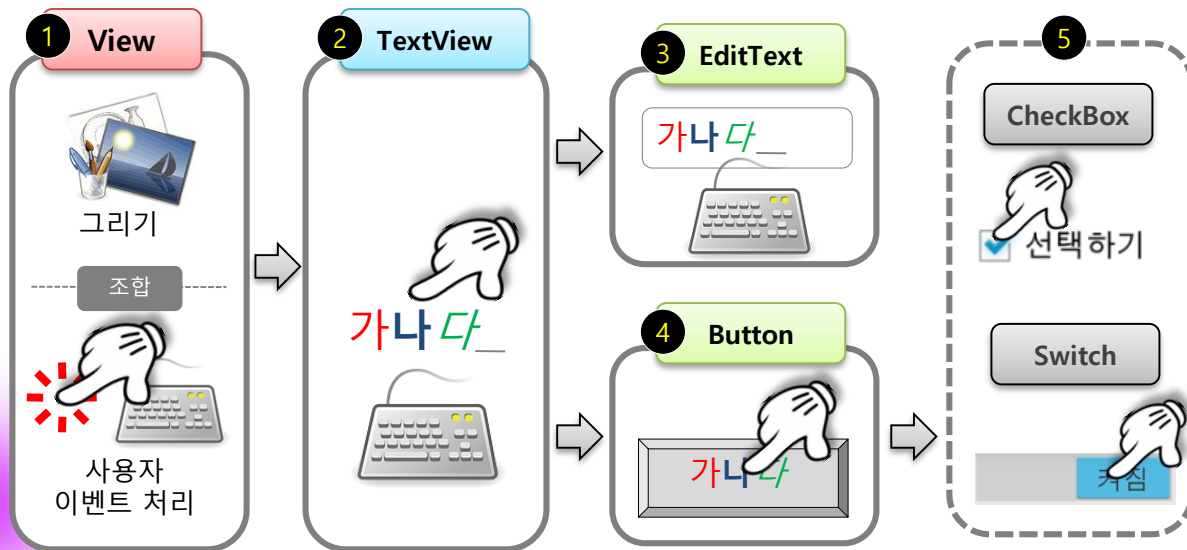
1. 뷰의 개요 ▶ 뷰와 뷰그룹

❖ View 클래스 계층도



■ 안드로이드에서 제공하는 뷰는 총 82개 가량된다.

상속 관계만으로도 각 뷰들의 특징을 쉽게 이해할 수 있다.



이처럼 나머지 뷰들도 최상위 뷰를 기점으로 한 단계 혹은 많아야 두 단계의 상속된 뷰 특징만 이해하면 된다. 반대로 특정 뷰를 쉽게 익히려면 반드시 상속 관계를 먼저 확인해야 한다.



1. 뷰의 개요 ▶ 뷰와 뷰그룹

❖ 클래스 상속관계 찾기

- <http://developer.android.com/reference/packages>에서 자세히 확인할 수 있음

The screenshot shows the Android Developer website's reference section for the `Button` class. The left sidebar lists various Android classes, with `Button` highlighted. The main content area displays the class hierarchy for `Button`, showing it extends `TextView`, which extends `android.view.View`, which extends `android.widget.TextView`, which finally extends `android.widget.Button`. Below the hierarchy, there are sections for 'Known direct subclasses' (listing `CompoundButton`) and 'Known indirect subclasses' (listing `CheckBox`, `RadioButton`, `Switch`, and `ToggleButton`). A description of the `Button` class is provided, along with an example of how to display a button in an activity's layout XML file.

```
public class Button
extends TextView
java.lang.Object
↳ android.view.View
↳ android.widget.TextView
↳ android.widget.Button
```

Known direct subclasses

- CompoundButton

Known indirect subclasses

- CheckBox, RadioButton, Switch, ToggleButton

A user interface element the user can tap or click to perform an action.

To display a button in an activity, add a button to the activity's layout XML file:

```
<Button
    android:id="@+id/button_id"
    android:layout_height="wrap_content"
    android:layout_width="wrap_content"
    android:text="@string/self_destruct" />
```

To specify an action when the button is pressed, set a click listener on the button object in the corresponding activity

그림 4-4 클래스의 상속 관계를 찾는 방법



1. 뷰의 개요 ▶ View 클래스의 XML 속성

❖ View 클래스로부터 상속받은 Button의 XML 속성

```
<Button  
    android:id="@+id/btn1"  
    android:layout_width="wrap_content"  
    android:layout_height="wrap_content"  
    android:background="#ff0000"  
    android:text="버튼입니다"  
>
```




XML 레이아웃의 구성

- 뷰 태그와 속성으로 구성됨

<시작 태그

속성1="속성값1"

속성2="속성값2"

...

</끝 태그>

<TextView

android:layout_width="wrap_content"

android:layout_height="wrap_content"

android:text="Hello World!" />

속성Prefix : 속성명1="속성값1"

android:layout_width="wrap_content"

app:layout_constraintLeft_toLeftOf="parent"

<android.support.constraint.ConstraintLayout

android:layout_width="match_parent"

android:layout_height="match_parent"

...

</android.support.constraint.ConstraintLayout>



1. 뷰의 개요 ▶ View 클래스의 XML 속성

❖ id 속성

- ✓ id 속성은 모든 위젯의 아이디를 나타냄
 - ✓ Java 코드에서 위젯에 접근할 때 id 속성에 지정한 아이디를 사용
 - ✓ id 속성은 위젯에 아이디를 새로 부여하는 개념이므로 “@+id/” 형식으로 지정
 - ✓ / 다음에는 새로 지정할 id를 적음
- android:id="@+id/btn1"의 의미는 버튼 위젯의 아이디를 btn1로 부여한 것

```
위젯 변수 = (위젯 형) findViewById(R.id.위젯id);
```



```
Button button1;  
button1 = (Button) findViewById(R.id.btn1);
```



뷰의 ID 속성

```
<Button  
  android:id="@+id/button"  
  android:layout_width="match_parent"  
  android:layout_height="match_parent"  
  android:text="Layout"  
>
```

[버튼의 id 추가]

```
public class SampleLayoutActivity extends Activity  
{  
  public void onCreate(Bundle savedInstanceState)  
  {  
    super.onCreate(savedInstanceState);  
    setContentView(R.layout.main);  
    Button button = (Button)  
        findViewById(R.id.button);  
  }  
}
```

[버튼의 id 참조]

• 인플레이션(Inflation)

- XML 레이아웃에 정의된 정보를 메모리 상에서 객체로 만드는 객체화 과정
- 애플리케이션이 시작될 때 이 과정을 거쳐 메모리 상에 만들어진 객체들을 참조하기 위해 ID를 지정함

• id 속성은 자바 코드 상에서 R.id.[ID]와 같은 형태로 참조함("@+id/...")

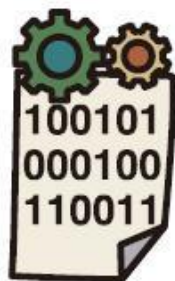


뷰의 ID 속성

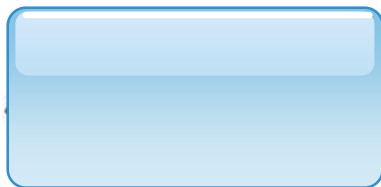
자바 파일 (.java)



XML 파일 (.xml)



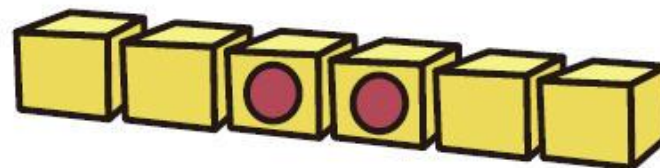
클래스 파일 (.class)



실제 버튼



찾아서 기능 부여
(find)



메모리의 버튼

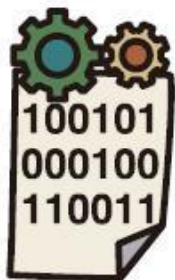


소스와 XML이 분리되면서 화면에 추가한 위젯 찾기

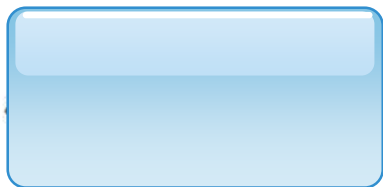
자바 파일 (.java)



XML 파일 (.xml)



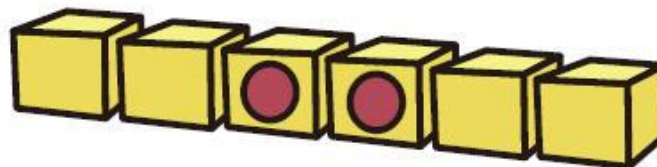
클래스 파일 (.class)



실제 버튼



찾아서 기능 부여
(find)



메모리의 버튼

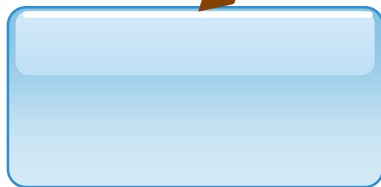


자바 소스에서 버튼을 만들고 메모리에 있는 버튼 가리키기

- XML에 버튼을 넣어두지 않고 소스 코드에서 직접 버튼을 만들 수도 있음
- 그렇게 하면 메모리에 버튼이 만들어지고 그 버튼을 가리키게 됨

자바 소스

```
Button button1 = new Button();
```



실제 버튼

기능 부여

메모리의 버튼



findViewById 앞에 코드 추가

- 버튼을 찾아 버튼 변수라는 것을 이용해 가리킴

```
@Override
```

```
protected void onCreate(Bundle savedInstanceState) {
```

```
    super.onCreate(savedInstanceState);
```

```
    setContentView(R.layout.activity_main);
```

```
    // (1) 버튼 변수로 받기
```

```
    Button button2 = (Button) findViewById(R.id.button2);
```

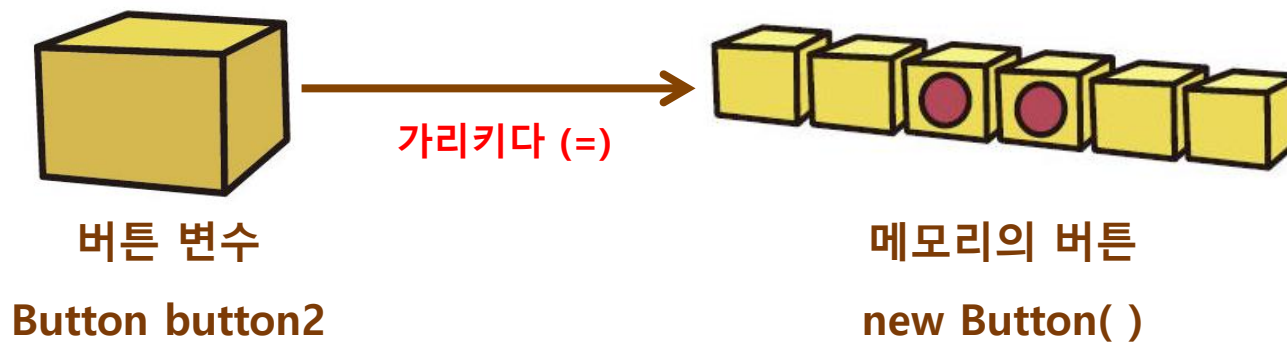
```
}
```



XML에서 찾은 버튼을 변수로 가리키기

- = : '가리키다' 의 의미

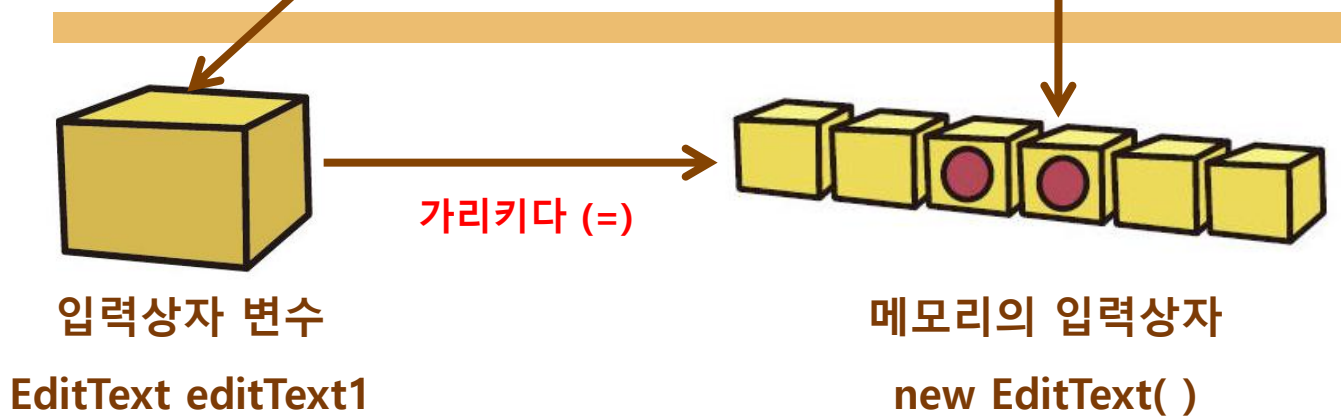
실행



코드에서 직접 만든 입력상자를 가리키기

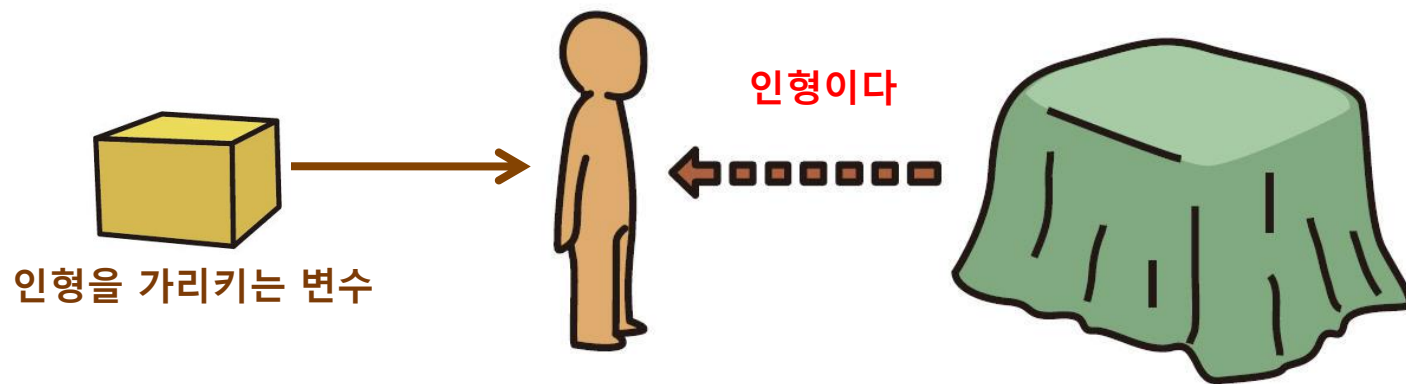
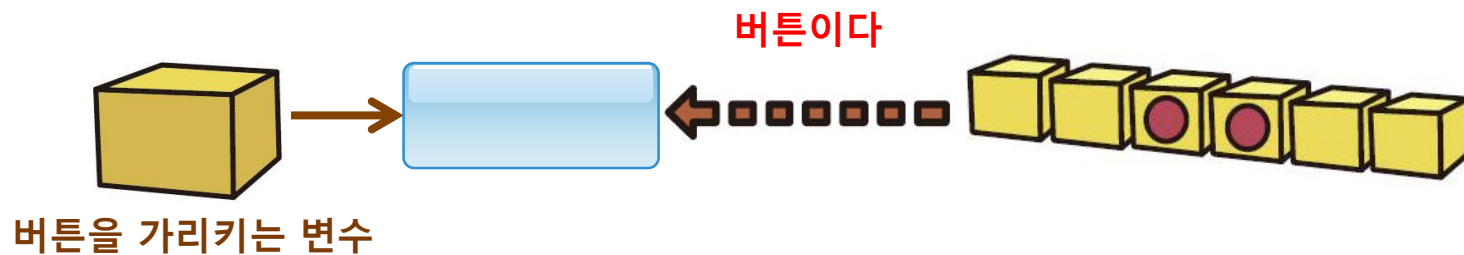
자바 소스

```
EditText editText1 = new EditText();
```





형 변환하여 변수로 가리키기





클릭했을 때의 기능 넣기 (1)

The screenshot shows an IDE window with two tabs: `activity_main.xml` and `*MainActivity.java`. The `MainActivity.java` file contains the following code:

```
package org.androidtown.basic.hello;

import android.app.Activity;
import android.os.Bundle;
import android.widget.Button;

public class MainActivity extends Activity {

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);

        Button button2 = (Button) findViewById(R.id.button2);
        button2.setOn

    }
}
```

A dropdown menu is open below `button2.setOn`, listing various listener methods:

- `setOnClickListener(OnClickListener l) : void - View`
- `setOnCreateContextMenuListener(OnCreateContextMenuListener l) : void - View`
- `setOnDragListener(OnDragListener l) : void - View`
- `setOnEditorActionListener(OnEditorActionListener l) : void - View`
- `setOnFocusChangeListener(OnFocusChangeListener l) : void - View`
- `setOnGenericMotionListener(OnGenericMotionListener l) : void - View`
- `setOnHoverListener(OnHoverListener l) : void - View`
- `setOnKeyListener(OnKeyListener l) : void - View`
- `setOnLongClickListener(OnLongClickListener l) : void - View`
- `setOnSystemUiVisibilityChangeListener(OnSystemUiVisibilityChangeListener l) : void - View`
- `setOnTouchListener(OnTouchListener l) : void - View`

On the right side of the IDE, a tooltip for `setOnClickListener` is displayed:

Register a callback to be invoked when this view is clicked. If this view is not clickable, it becomes clickable.

Parameters:

- l The callback that will run

See Also:

- [setClickable\(boolean\)](#)

Press 'Ctrl+Space' to show Template Proposals



클릭했을 때의 기능 넣기 (2)

```
activity_main.xml  MainActivity.java
package org.androidtown.basic.hello;

import android.app.Activity;
import android.os.Bundle;
import android.widget.Button;

public class MainActivity extends Activity {

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);

        Button button2 = (Button) findViewById(R.id.button2);
        button2.setOnClickListener(new OnClickListener() {
            // 클릭했을 때의 기능 넣기
        });
    }
}
```



클릭했을 때의 기능 넣기 (3)

```
activity_main.xml *MainActivity.java

import android.app.Activity;
import android.os.Bundle;
import android.view.View;
import android.view.View.OnClickListener;
import android.widget.Button;

public class MainActivity extends Activity {

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);

        Button button2 = (Button) findViewById(R.id.button2);
        button2.setOnClickListener(new OnClickListener() {

            @Override
            public void onClick(View v) {
                // TODO Auto-generated method stub
            }
        });
    }
}
```



클릭했을 때의 기능 넣기 (4)

- Toast. 까지 입력한 상태

```
activity_main.xml *MainActivity.java
import android.app.Activity;
import android.os.Bundle;
import android.view.View;
import android.view.View.OnClickListener;
import android.widget.Button;
import android.widget.Toast;

public class MainActivity {

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);

        Button button2 = findViewById(R.id.button2);
        button2.setOnClickListener(new OnClickListener() {

            @Override
            public void onClick() {
                Toast.makeText(getApplicationContext(), "버튼이 눌렸어요.", Toast.LENGTH_SHORT).show();
            }
        });
    }
}
```

Show the view or text notification for a long period of time. This time could be user-definable.
See Also: [setDuration](#)

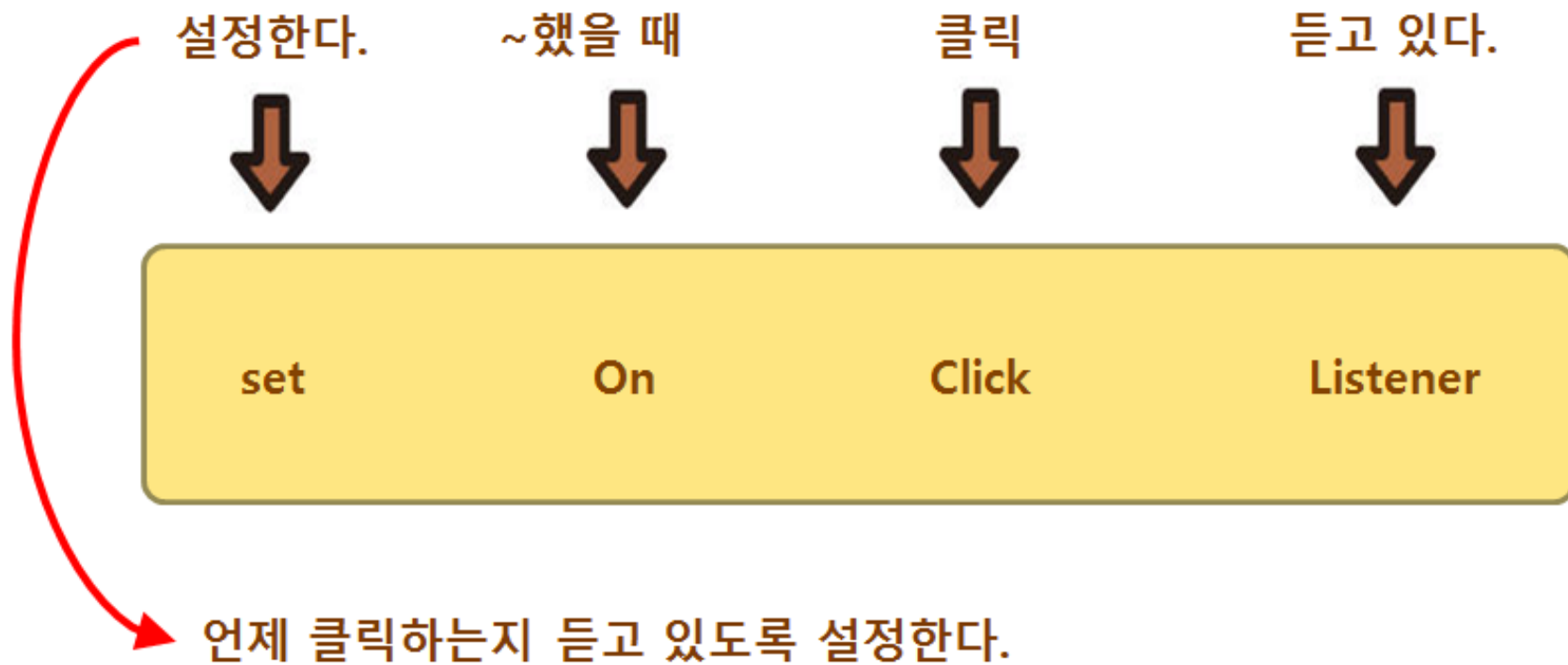
- LENGTH_LONG : int - Toast
- LENGTH_SHORT : int - Toast
- class : Class<android.widget.Toast>
- makeText(Context context, CharSequence text, int duration, boolean gravity): void
- makeText(Context context, int resId, int duration, boolean gravity): void

Press 'Tab' from proposal table or click for focus

Press 'Ctrl+Space' to show

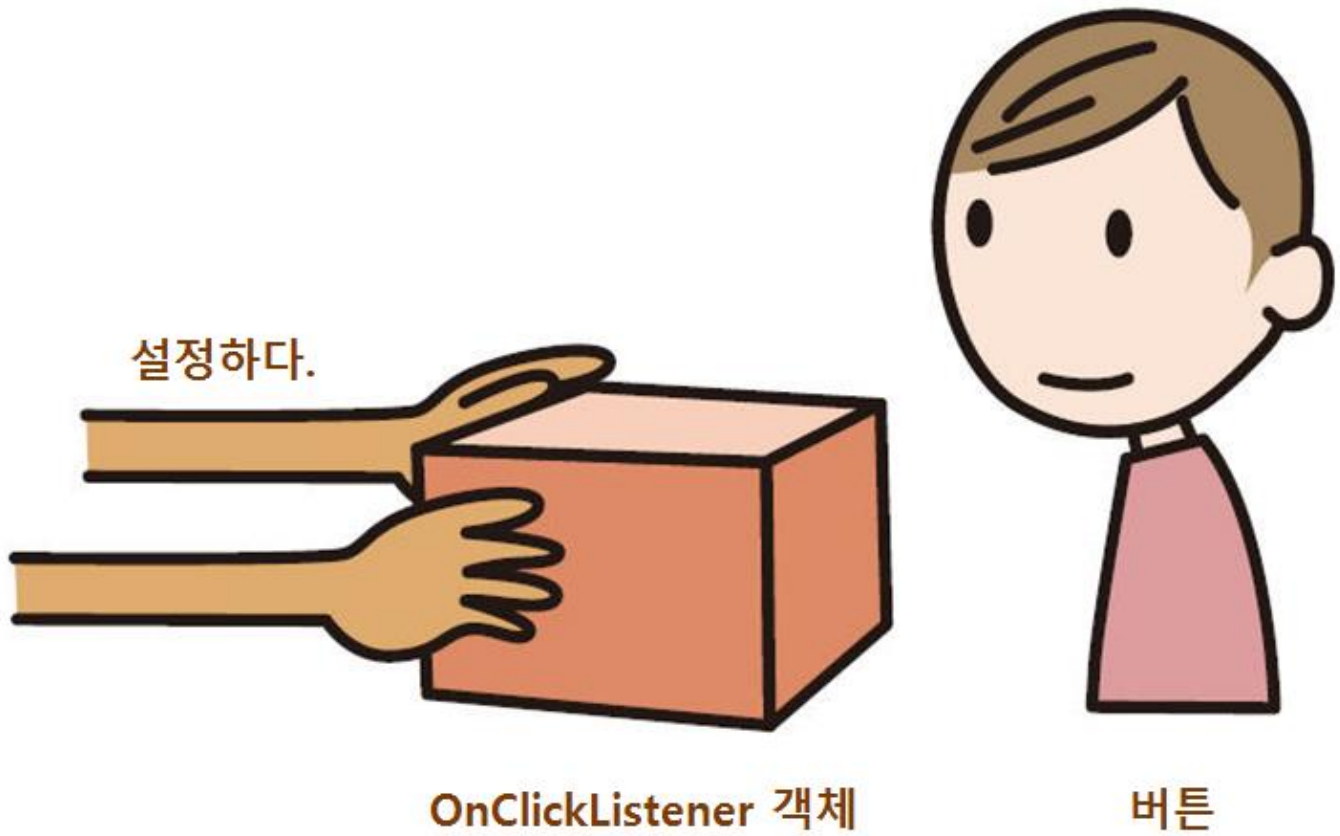


setOnClickListener 라는 말의 뜻



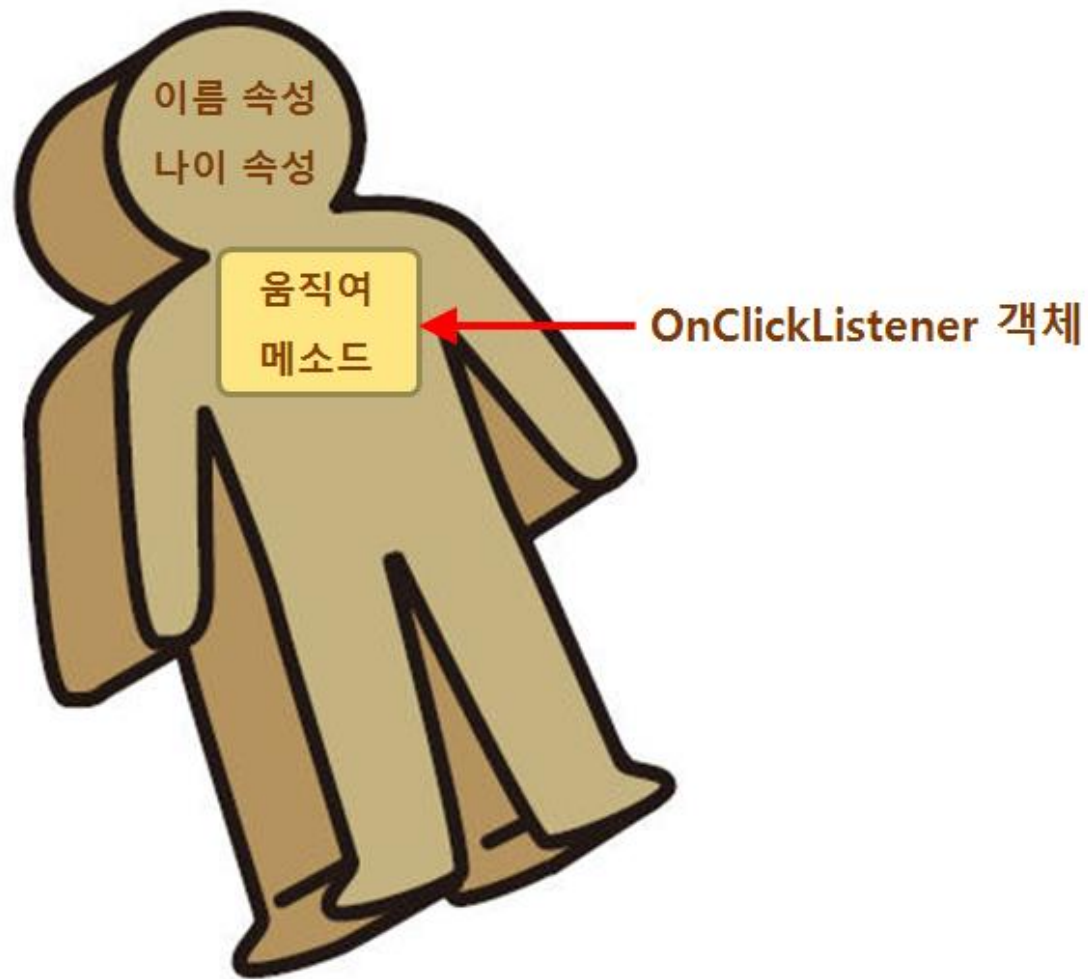


버튼에 OnClickListener 객체를 설정한다는 의미



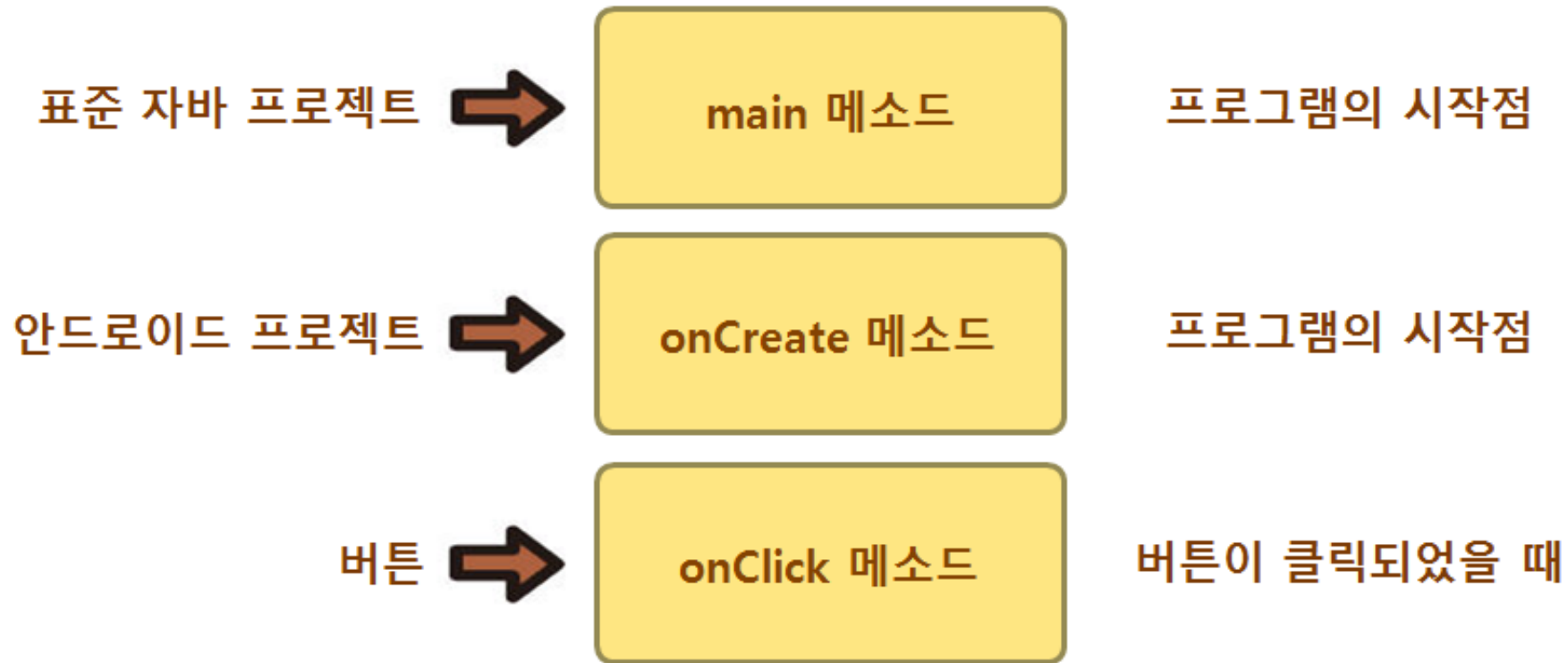


객체가 가질 수 있는 속성과 메소드



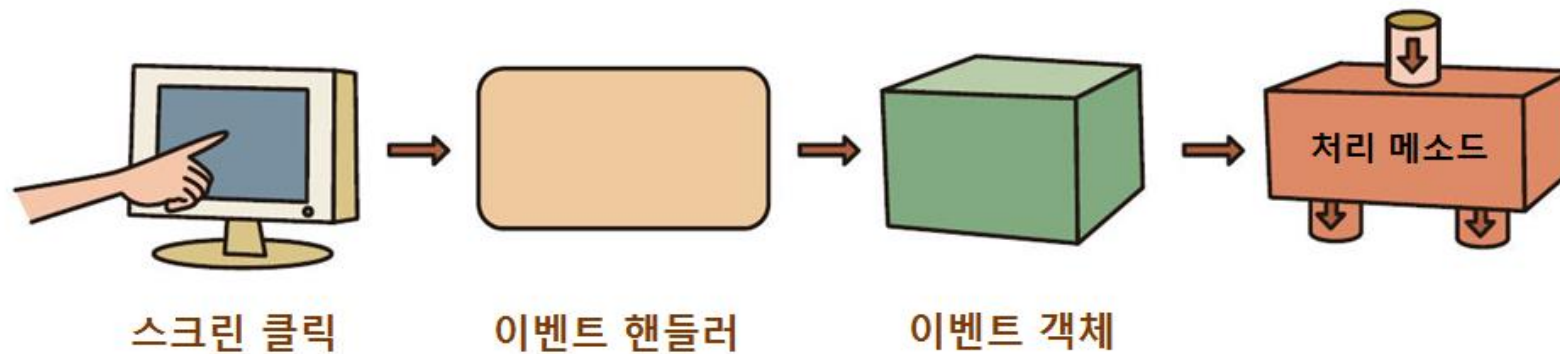
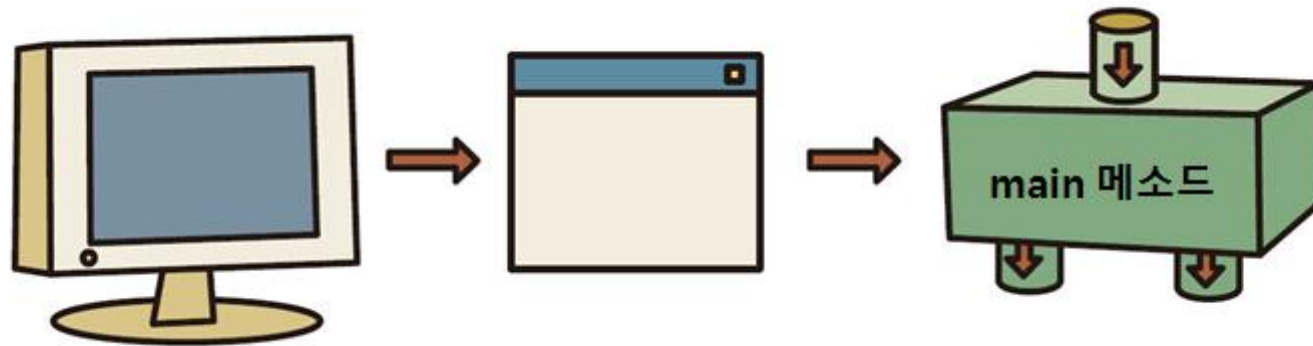


호출하지도 않은 메소드가 알아서 호출되는 경우

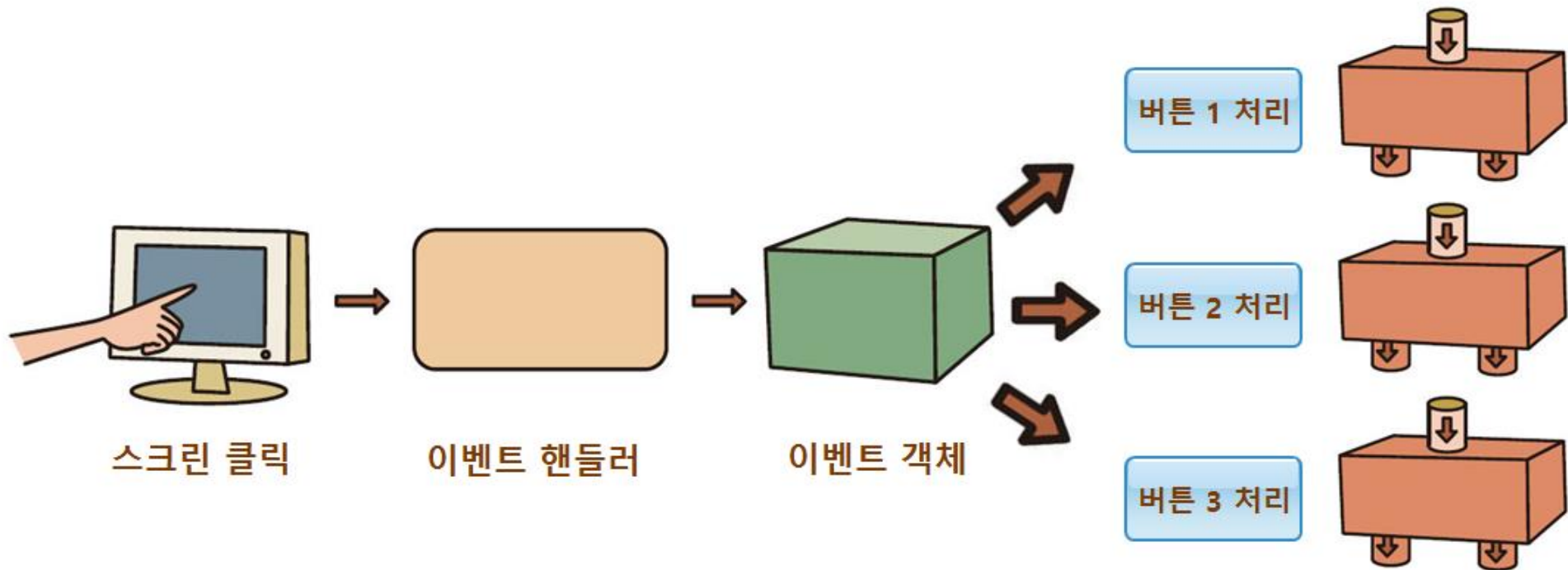




콜백 메소드가 동작하는 방식과 클릭했을 때의 동작



화면에 있는 각각의 버튼으로 이벤트 전달하기





1. 뷰의 개요 ▶ View 클래스의 XML 속성

❖ id 속성 예제

예제 4-1 id 속성의 XML 코드

```
1 <LinearLayout    ~~~ 중간 생략 ~~~
2     android:orientation="vertical" >
3     <TextView
4         android:id="@+id/textView1"
5         android:layout_width="wrap_content"
6         android:layout_height="wrap_content"
7         android:text="성별 선택" />
8     <RadioButton
9         android:id="@+id/female"
10        android:layout_width="wrap_content"
11        android:layout_height="wrap_content"
12        android:text="여성" />
13    <RadioButton
14        android:id="@+id/male"
15        android:layout_width="wrap_content"
16        android:layout_height="wrap_content"
17        android:text="남성" />
18 </LinearLayout>
```

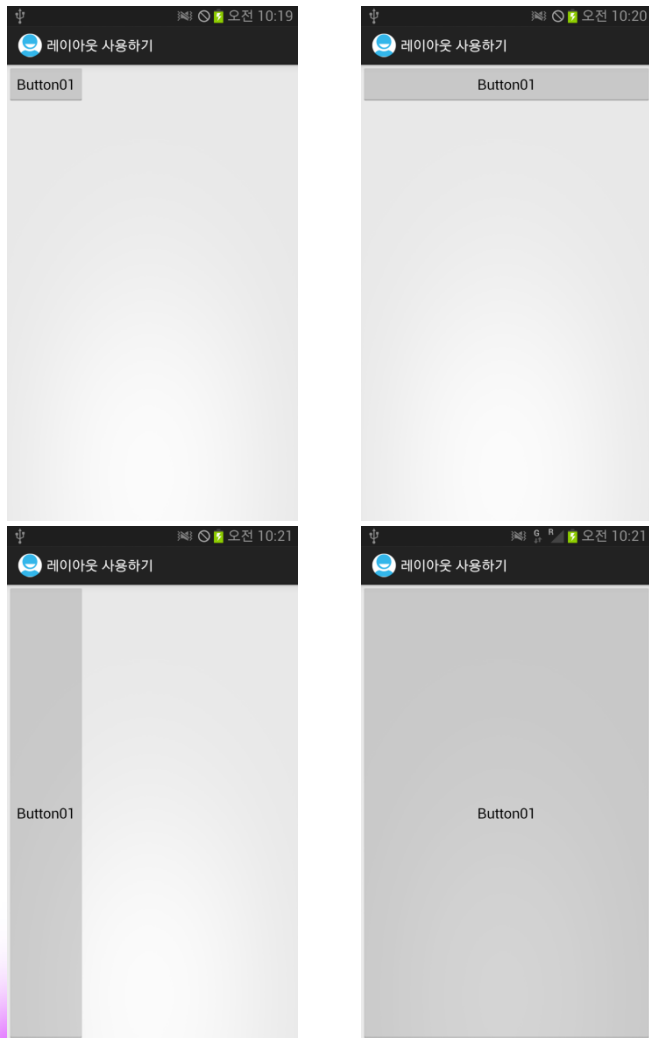
성별 선택

☐ 여성

☒ 남성



뷰의 대표적인 속성



[match_parent와 wrap_content 값을 폭과 넓이에 적용한 예]

- **layout_width, layout_height**

- 가장 기본적인 속성으로 뷰의 폭과 높이를 설정함

- (1) **match_parent**

- 무조건 남아 있는 여유 공간을 채움,

- 속성값 match_parent(=fill_parent)는 이름 그대로 자신의 부모에 폭이나 높이를 맞춘다는 의미

- (2) **wrap_content**

- 뷰에 들어 있는 내용물의 크기에 따라 뷰의 크기가 결정됨

- 자신의 폭이나 높이를 자신 안의 글자가 꼭 들어갈 정도로 한다는 의미

- (3) **크기 값 지정**

- 크기를 고정된 값으로 직접 지정하고 싶을 때 사용함

- ex) "100px", "200dp"

- **background**

- 뷰의 배경을 설정함 (배경색, 배경 이미지 등)

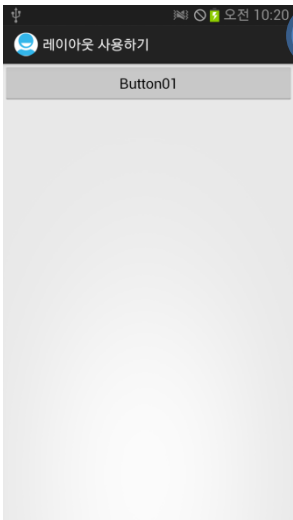


XML 레이아웃으로 구성하기

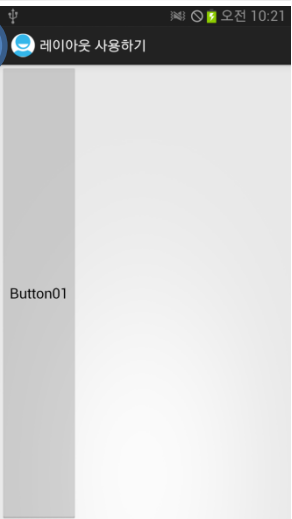
1



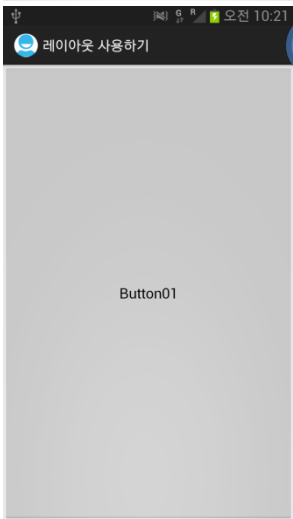
2



3



4



[match_parent와 wrap_content 값을 폭과 넓이에 적용한 예]

1

```
<Button
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="Layout"
/>
```

2

```
<Button
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:text="Layout"
/>
```

3

```
<Button
    android:layout_width="wrap_content"
    android:layout_height="match_parent"
    android:text="Layout"
/>
```

4

```
<Button
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:text="Layout"
/>
```

- match_parent는 예전 버전에서는 fill_parent 도 사용 가능



뷰의 background 속성

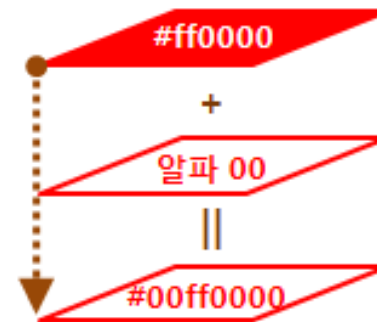
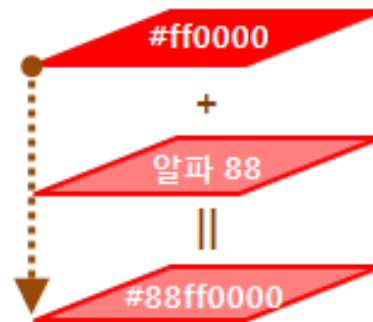
[Format]

#RGB

#ARGB

#RRGGBB

#AARRGGBB



[배경색에 알파값을 적용하여 투명도를 조절하는 경우]

- XML 레이아웃에서 색상을 지정할 때는 '#' 기호를 앞에 붙인 후, ARGB(A : Alpha, R : Red, G : Green, B : Blue)의 순서대로 색상의 값을 기록함
- 16진수 값을 지정할 때는 여러 가지 포맷을 사용할 수 있음



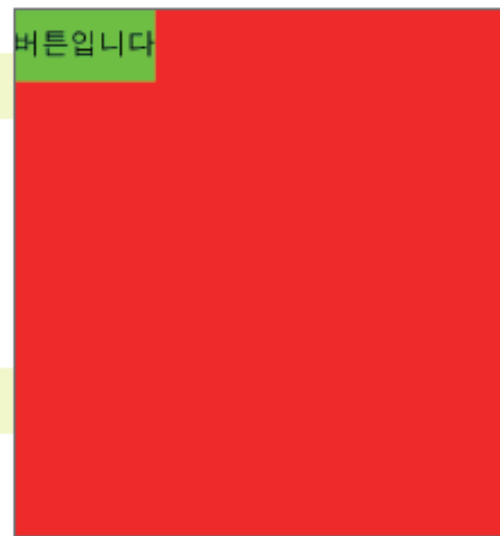
1. 뷰의 개요 ▶ View 클래스의 XML 속성

❖ background 속성

- ✓ background 속성은 위젯의 색상을 주로 #RRGGBB 값으로 지정

예제 4-7 background 속성의 XML 코드

```
1 <LinearLayout
2     android:background="#ff0000"
3     ~~~~ 중간 생략 ~~~~ >
4     <Button
5         android:layout_width="wrap_content"
6         android:layout_height="wrap_content"
7         android:background="#00ff00"
8         android:text="버튼입니다" />
9 </LinearLayout>
```





뷰의 크기 지정에 사용되는 단위

단위	단위 표현	설 명
px	픽셀	화면 픽셀
dp 또는 dip	밀도 독립적 픽셀 (density independent pixel)	160dpi 화면을 기준으로 한 픽셀 예) 1인치 당 160개의 점이 있는 디스플레이 화면에서 1dp는 1px와 같음. 1인치 당 320개의 점이 있는 디스플레이 화면에서 1dp는 2px와 같음.
sp 또는 sip	축척 독립적 픽셀 (scale independent pixel)	가변 글꼴을 기준으로 한 픽셀로 dp와 유사하나 글꼴의 설정에 따라 달라짐
in	인치	1인치로 된 물리적 길이
mm	밀리미터	1밀리미터로 된 물리적 길이
em	텍스트 크기	글꼴과 상관없이 동일한 텍스트 크기 표시



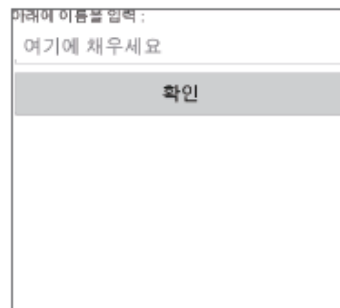
1. 뷰의 개요 ▶ View 클래스의 XML 속성

❖ padding 속성

- ✓ Padding은 위젯의 경계선으로부터 위젯 안의 요소가 떨어지도록 설정

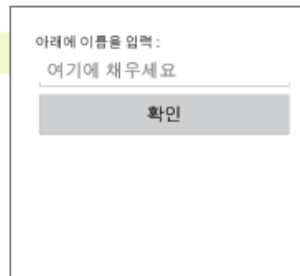
예제 4-10 간격이 없는 XML 코드

```
1 <LinearLayout >
2   <TextView
3     android:text="아래에 이름을 입력 : " />
4   <EditText
5     android:hint="여기에 채우세요" />
6   <Button
7     android:text="확인" />
8 </LinearLayout>
```



예제 4-11 padding 속성의 XML 코드

```
1 <LinearLayout
2   android:padding="30dp" >
3   <TextView
4     android:text="아래에 이름을 입력 : " />
5   <EditText
6     android:hint="여기에 채우세요" />
7   <Button
8     android:text="확인" />
9 </LinearLayout>
```





1. 뷰의 개요 ▶ View 클래스의 XML 속성

❖ layout_margin 속성

- ✓ 위젯과 위젯 사이에 여유를 두고 싶다면 layout_margin 속성을 사용

예제 4-12 layout_margin 속성의 XML 코드

```
1 <LinearLayout
2     android:padding="30dp" >
3     <TextView
4         android:layout_margin="20dp"
5         android:text="아래에 이름을 입력 :" />
6     <EditText
7         android:layout_margin="20dp"
8         android:hint="여기에 채우세요" />
9     <Button
10        android:layout_margin="20dp"
11        android:text="확인" />
12 </LinearLayout>
```

아래에 이름을 입력 :

여기에 채우세요

확인



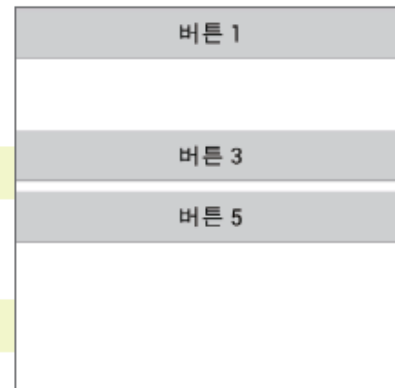
1. 뷰의 개요 ▶ View 클래스의 XML 속성

❖ visibility 속성

- ✓ visibility 속성은 위젯을 보일 것인지 여부를 결정
- ✓ 디폴트인 visible은 보이는 상태, invisible과 gone은 안 보이는 상태

예제 4-13 visibility 속성의 XML 코드

```
1 <Button
2     android:text="버튼 1" />
3 <Button
4     android:visibility="invisible"
5     android:text="버튼 2" />
6 <Button
7     android:visibility="visible"
8     android:text="버튼 3" />
9 <Button
10    android:visibility="gone"
11    android:text="버튼 4" />
12 <Button
13    android:text="버튼 5" />
```





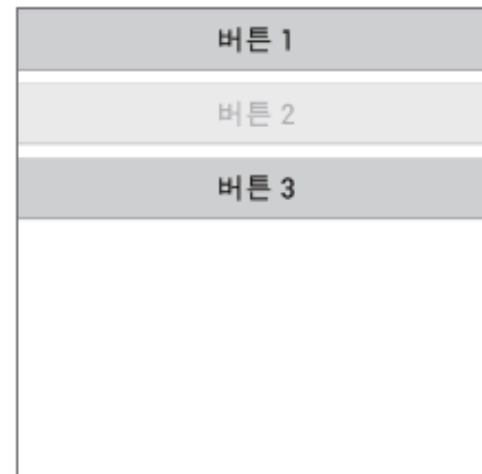
1. 뷰의 개요 ▶ View 클래스의 XML 속성

❖ enabled, clickable 속성

- ✓ enabled 속성 : 위젯의 동작 여부
- ✓ clickable 속성 : 클릭이나 터치가 가능하도록 함
- ✓ true와 false로 지정(디폴트: true)

예제 4-14 enabled 및 clickable 속성의 XML 코드

```
1 <Button
2     android:text="버튼 1" />
3 <Button
4     android:enabled="false"
5     android:text="버튼 2" />
6 <Button
7     android:clickable="false"
8     android:text="버튼 3" />
```





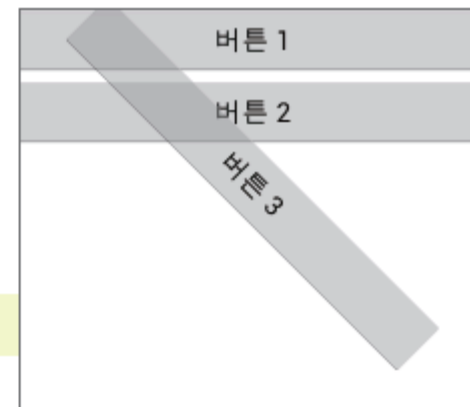
1. 뷰의 개요 ▶ View 클래스의 XML 속성

❖ rotation 속성

- ✓ rotation은 위젯을 회전시켜서 출력
- ✓ 값은 각도로 지정

예제 4-15 rotation 속성의 XML 코드

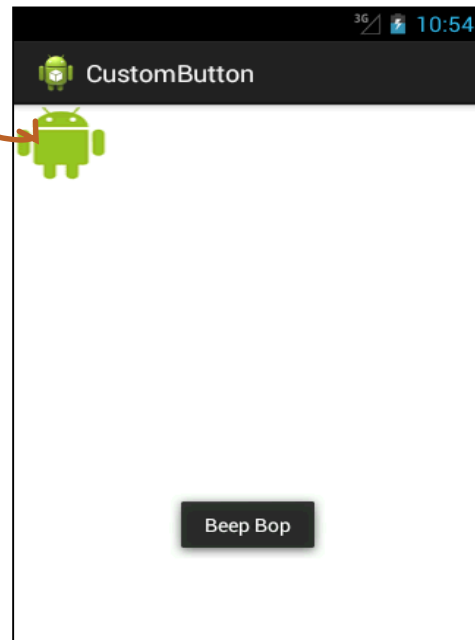
```
1 <Button  
2     android:text="버튼 1" />  
3 <Button  
4     android:text="버튼 2" />  
5 <Button  
6     android:rotation="45"  
7     android:text="버튼 3" />
```





1. 뷰의 개요 ▶ 커스텀 버튼(상태 드로어블)

- 버튼 위에 텍스트 대신에 이미지가 그려져 있는 버튼
- 버튼을 누르면 눌려지는 효과를 발생하는 버튼





1. 뷰의 개요 ▶ 커스텀 버튼(상태 드로어블)

- XML로 버튼에 사용되는 이미지를 등록한다.

/res/drawable/android_button.xml

```
<?xml version="1.0" encoding="utf-8"?>
<selector xmlns:android="http://schemas.android.com/apk/res/android">
    <item android:drawable="@drawable/android_pressed"
        android:state_pressed="true" />
    <item android:drawable="@drawable/android_normal" />
</selector>
```





1. 뷰의 개요 ▶ 커스텀 버튼(상태 드로어블)

- 레이아웃 파일에 버튼을 정의한다.

/res/layout/main.xml

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical" >

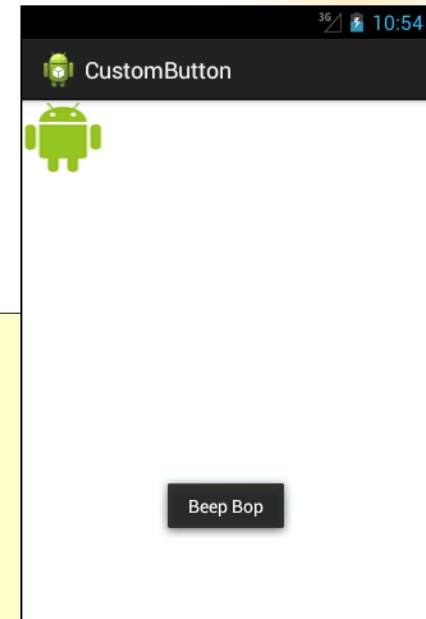
    <Button
        android:id="@+id/button"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:background="@drawable/android_button"
        android:padding="10dp" />

</LinearLayout>
```



1. 뷰의 개요 ▶ 커스텀 버튼(상태 드로어블)

```
public class MainActivity extends Activity {  
    @Override  
    public void onCreate(Bundle savedInstanceState) {  
        super.onCreate(savedInstanceState);  
        setContentView(R.layout.main);  
        Button button = (Button) findViewById(R.id.button);  
        button.setOnClickListener(new OnClickListener() {  
            public void onClick(View v) {  
                Toast.makeText(getApplicationContext(), "Beep Bop",  
                    Toast.LENGTH_SHORT).show();  
            }  
        });  
    }  
}
```





2. 기본 위젯 다루기 ▶ 텍스트뷰

❖ 계층도

```
java.lang.Object
└─ android.view.View
    └─ android.widget.TextView
```

TextView 계층도

- ✓ **textColor** 속성 : 글자의 색상을 지정, #RRGGBB나 #AARRGGBB 형식
- ✓ **textSize** 속성 : 글자의 크기를 dp, px, in, mm, sp 단위로 지정
- ✓ **typeface** 속성 : 글자의 글꼴을 지정
 - 값으로 sans, serif, monospace 설정, 디폴트는 normal
- ✓ **textStyle** 속성 : 글자의 스타일을 지정
 - 값으로 bold, italic, bold|italic을 설정, 디폴트는 normal
- ✓ **singleLine** 속성 : 글이 길어 줄이 넘어갈 경우 강제로 한 줄만 출력하고 문자열의 맨 뒤에 '...'를 표시.
 - 값으로 true와 false를 설정할 수 있으며 디폴트는 false



2. 기본 위젯 다루기 ▶ 텍스트뷰

■ textSize 속성

텍스트 크기를 설정하는 속성이다.

■ textStyle 속성

텍스트의 두 가지 스타일을 지정하는 속성이다.

- bold : 글자를 두껍게 표시한다.
- italic : 글자를 기울여서 표시한다

두 가지 속성값은 '|' 기호를 이용하여 조합하여 사용할 수도 있다.

■ typeface 속성

텍스트의 글자체를 설정하는 속성이다. 설정 값은 아래와 같이 세 가지가 존재한다.

- sans : 글자의 굵기가 같고 장식이 없는 글자체
- serif : 글자의 굵기가 다르고 장식이 있는 글자체
- monospace : 고정 폭의 글자체





2. 기본 위젯 다루기 ▶ 텍스트뷰

❖ 텍스트뷰 예제

예제 4-16 글자 관련 속성의 XML 코드

```
1 <TextView
2     android:textSize="30dp"
3     android:text="textSize 속성" />
4 <TextView
5     android:textSize="30dp"
6     android:textColor="#00FF00"
7     android:text="textColor 속성" />
8 <TextView
9     android:textSize="30dp"
10    android:textStyle="bold|italic"
11    android:text="textStyle 속성" />
12 <TextView
13    android:textSize="30dp"
14    android:typeface="serif"
15    android:text="typeface 속성" />
16 <TextView
17    android:textSize="30dp"
18    android:singleLine="true"
19    android:text="singleLine 속성 singleLine 속성 singleLine 속성" />
```

textSize 속성
textColor 속성
textStyle 속성
typeface 속성
singleLine 속성 single...



2. 기본 위젯 다루기 ▶ Java코드로 XML 속성 설정

❖ 텍스트뷰 속성을 변경하는 예제

예제 4-17 텍스트뷰가 3개 있는 activity_main.xml

```
1 <TextView
2     android:text="TextView 연습 1"
3     android:id="@+id/textView1" />
4 <TextView
5     android:text="TextView 연습 2"
6     android:id="@+id/textView2" />
7 <TextView
8     android:text="TextView 연습 3"
9     android:id="@+id/textView3" />
```

TextView 연습 1
TextView 연습 2
TextView 연습 3



2. 기본 위젯 다루기 ▶ Java코드로 XML 속성 설정

❖ 텍스트뷰 속성을 변경하는 Java 코드

예제 4-18 텍스트 속성을 변경하는 Java 코드

```
1 public void onCreate(Bundle savedInstanceState) {  
2     super.onCreate(savedInstanceState);  
3     setContentView(R.layout.main);  
4  
5     TextView tv1, tv2, tv3;  
6     tv1 = (TextView) findViewById(R.id.textView1);  
7     tv2 = (TextView) findViewById(R.id.textView2);  
8     tv3 = (TextView) findViewById(R.id.textView3);  
9  
10    tv1.setText("안녕하세요?");  
11    tv1.setTextColor(Color.RED);  
12    tv2.setTextSize(30);  
13    tv2.setTypeface(Typeface.SANS_SERIF, Typeface.BOLD_ITALIC);  
14    tv3.setText("가나다라마바사아자차카타파하가나다라마바사아자차카타파하");  
15    tv3.setSingleLine();  
16 }
```

안녕하세요?

TextView 연습 2

가나다라마바사아자차카타파하가나다라마바사아자차카타파하



2. 기본 위젯 다루기 ▶ 텍스트뷰

res/layout/textview_font.xml

```
<LinearLayout xmlns:android="..."
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:layout_margin="10dp"
    android:orientation="vertical">

    <TextView android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Hello TextView"
        android:textColor="#F00"
        android:textSize="20dp"
        android:textStyle="normal"
        android:typeface="sans" />

    <EditText android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Hello EditText"
        android:textColor="#0F0"
        android:textSize="30dp"
        android:textStyle="italic"
        android:typeface="serif" />

    <Button android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Hello TextView"
        android:textColor="#00F"
        android:textSize="40dp"
        android:textStyle="bold"
        android:typeface="monospace" />

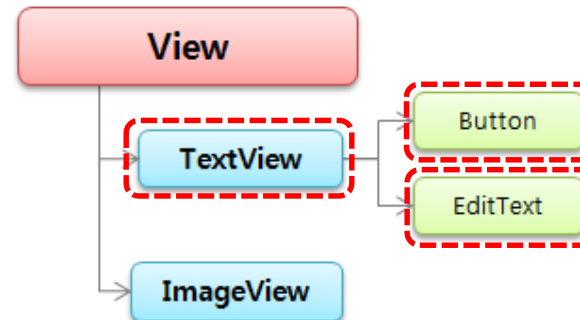
</LinearLayout>
```

 View

Hello TextView

Hello EditText

Hello
TextView





2. 기본 위젯 다루기 ▶ 텍스트뷰

res/layout/textview_single_line.xml

```
<LinearLayout xmlns:android="..."
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:layout_margin="10dp"
    android:orientation="vertical">

    <TextView android:layout_width="200dp"
        android:layout_height="wrap_content"
        android:layout_marginBottom="20dp"
        android:background="#FF0"
        android:text="동해물과 백두산이 마르고 닳도록 하느님이"/>

    <TextView android:layout_width="200dp"
        android:layout_height="wrap_content"
        android:background="#FF0"
        android:text="동해물과 백두산이 마르고 닳도록 하느님이"/>
</LinearLayout>
```



View

동해물과 백두산이 마르고
닳도록 하느님이

동해물과 백두산이 마르고 닳...



View

동해물과 백두산이 마르고
닳도록 하느님이

공간이
남았는데도
내려섬

동해물과 백두산이 마르고 닳...

■ 워드래핑 Word Wrapping 기능

텍스트뷰는 기본적으로 단어가 잘리지 않으면서 줄바꿈을 하는 워드래핑 기능을 지원한다. 이는 텍스트뷰의 오류가 아니니 오해하지 말자.



2. 기본 위젯 다루기 ▶ 텍스트뷰

res/layout/textview_ellipsize.xml

```
<LinearLayout xmlns:android="..."
```

```
<TextView android:layout_width="200dp"
    android:layout_height="wrap_content"
    android:layout_marginBottom="20dp"
    android:background="#FF0"
    android:text="동해물과 백두산이 마르고 닳도록 하느님이"
    android:singleLine="true"
    android:ellipsize="start"/>
```

```
<TextView android:layout_width="200dp"
    android:layout_height="wrap_content"
    android:layout_marginBottom="20dp"
    android:background="#FF0"
    android:text="동해물과 백두산이 마르고 닳도록 하느님이"
    android:singleLine="true"
    android:ellipsize="middle"/>
```

```
<TextView android:layout_width="200dp"
    android:layout_height="wrap_content"
    android:layout_marginBottom="20dp"
    android:background="#FF0"
    android:text="동해물과 백두산이 마르고 닳도록 하느님이"
    android:singleLine="true"
    android:ellipsize="end"/>
```

```
<TextView android:layout_width="200dp"
    android:layout_height="wrap_content"
    android:background="#FF0"
    android:text="동해물과 백두산이 마르고 닳도록 하느님이"
    android:singleLine="true"
    android:ellipsize="marquee"/>
```

```
</LinearLayout>
```



View

...산이 마르고 닳도록 하느님이

동해물과 백두산...도록 하느님이

동해물과 백두산이 마르고 닳...

동해물과 백두산이 마르고 닳도...

■ ellipsize 속성값

- start : 문장의 앞 부분을 생략하고 생략기호 ...을 문장 앞에 배치
- middle : 문장의 중간 부분을 생략하고 생략기호 ...을 문장 중간에 배치
- end : 기본값이며, 문장의 끝 부분을 생략하고 생략기호 ...을 문장 끝에 배치
- marquee : 문장의 끝 부분을 생략하고 생략기호 ... 대신 문장의 끝을 점차 투명하게 보이도록 하는 페이드아웃 효과를 제공



2. 기본 위젯 다루기 ▶ 텍스트뷰

res/layout/textview_marquee_value_and_marquee_repeat_limit.xml

```
<LinearLayout xmlns:android="..."
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:layout_margin="10dp"
    android:orientation="vertical">

    <TextView android:id="@+id/textview"
        android:layout_width="200dp"
        android:layout_height="wrap_content"
        android:layout_marginBottom="20dp"
        android:background="#FF0"
        android:text="동해물과 백두산이 ..."
        android:singleLine="true"
        android:ellipsize="marquee"
        android:marqueeRepeatLimit="1"/>

</LinearLayout>
```

src/MainActivity.java

```
public class MainActivity extends Activity
{
    @Override
    protected void onCreate( Bundle savedInstanceState )
    {
        super.onCreate( savedInstanceState );

        setContentView(
            R.layout.textview_marquee_value_and_marquee_repeat_limit );

        TextView textView = (TextView) findViewById( R.id.textview );
        textView.setSelected( true );
    }
}
```

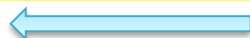
■ marqueeRepeatLimit 속성값

- 횟수 상수 : 글이 지정한 수만큼 반복해서 흐름
- marquee_forever : 글이 무한정 반복해서 흐름
- 만일 marqueeRepeatLimit 속성을 설정하지 않으면 기본적으로 3번 반복해서 흐른 뒤 멈춤



View

백두산이 마르고 닳도록 하느님

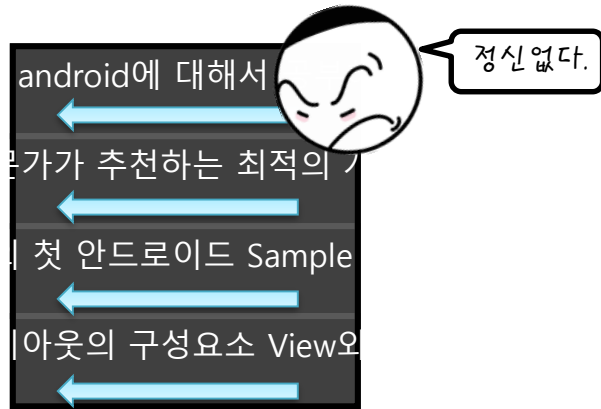


좌측으로 글이 흐른다.

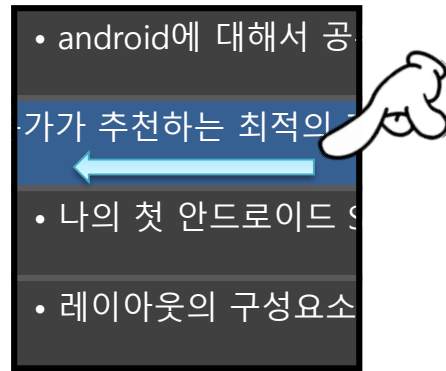


2. 기본 위젯 다루기 ▶ 텍스트뷰

■ 왜 텍스트뷰는 선택된 상태에서만 글이 흐를까?



리스트 환경에서 전체
아이템들이 모두 글이 흐른다.



리스트 환경에서 선택된
아이템만 글이 흐른다.



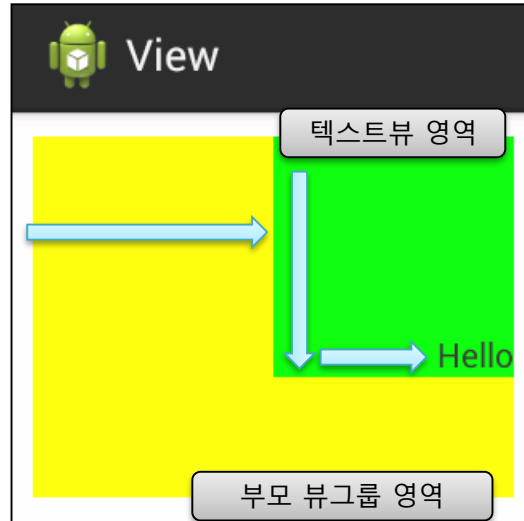
2. 기본 위젯 다루기 ▶ 텍스트뷰

res/layout/textview_gravity.xml

```
<LinearLayout xmlns:android="..."  
    android:layout_width="200dp"  
    android:layout_height="150dp"  
    android:layout_margin="10dp"  
    android:background="#FF0"  
    android:orientation="vertical">
```

```
    <TextView android:layout_width="100dp"  
        android:layout_height="100dp"  
        android:background="#0F0"  
        android:text="Hello"  
        android:layout_gravity="right"  
        android:gravity="bottom|right"/>
```

```
</LinearLayout>
```





2. 기본 위젯 다루기 ▶ 텍스트뷰

TextView 속성 - lineSpacingExtra, lineSpacingMultiplier

res/layout/textview_line_spacing_extra_and_multiplier.xml

```
<LinearLayout xmlns:android="..."
    android:layout_width="80dp"
    android:layout_height="match_parent"
    android:layout_margin="10dp"
    android:orientation="vertical">

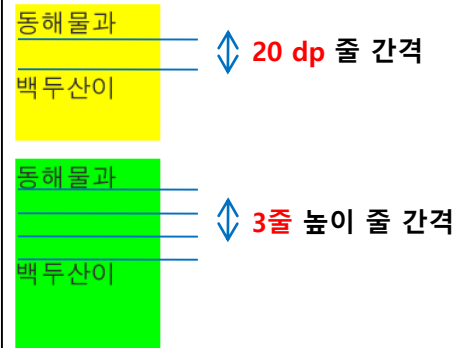
    <TextView android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_marginBottom="10dp"
        android:background="#FF0"
        android:text="동해물과 백두산이"
        android:lineSpacingExtra="20dp"/>

    <TextView android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_marginBottom="10dp"
        android:background="#0F0"
        android:text="동해물과 백두산이"
        android:lineSpacingMultiplier="3"/>

</LinearLayout>
```



View





2. 기본 위젯 다루기 ▶ 텍스트뷰

❖ XML 속성과 관련 메소드

표 4-1 XML 속성과 관련 메소드

XML 속성	관련 메소드	비고
background	setBackgroundColor()	View 클래스
clickable	setClickable()	View 클래스
focusable	setFocusable()	View 클래스
id	setId()	View 클래스
longClickable	setLongClickable()	View 클래스
padding	setPadding()	View 클래스
rotation	setRotation()	View 클래스
scaleX, scaleY	setScaleX(), setScaleY()	View 클래스
visibility	setVisibility()	View 클래스
gravity	setGravity()	TextView 클래스
inputType	setRawInputType()	TextView 클래스

표 4-1 XML 속성과 관련 메소드(계속)

XML 속성	관련 메소드	비고
password	setTransformationMethod()	TextView 클래스
text	setText()	TextView 클래스
textColor	setTextColor()	TextView 클래스
textSize	setTextSize()	TextView 클래스



2. 기본 위젯 다루기 ▶ 버튼과 에디트텍스트

❖ 버튼

- ✓ 사용자에게서 어떤 값을 입력받기 위한 가장 기본적인 위젯으로 활용도가 높음

<TextView

```
android:layout_width="match_parent"  
android:layout_height="wrap_content"  
android:text="나는 어떤 위젯일까요?" /a>
```



<Button

```
android:layout_width="match_parent"  
android:layout_height="wrap_content"  
android:text="나는 어떤 위젯일까요?" />
```



2. 기본 위젯 다루기 ▶ 버튼과 에디트텍스트

❖ 버튼 계층도

```
java.lang.Object
    ↳ android.view.View
        ↳ android.widget.TextView
            ↳ android.widget.Button
```

Button 계층도

❖ 버튼 XML과 Java 코드 작성 3단계

```
<Button
    android:id="@+id/button1"
    android:text="확인" />
```

① 버튼 변수 선언

```
Button mybutton;
```

② 변수에 버튼 위젯 대입

```
mybutton = (Button) findViewById(R.id.button1);
```

③ 버튼 클릭할 때 동작하는 클래스 정의

```
mybutton.setOnClickListener( new View.OnClickListener() {
    public void onClick(View v) {
        // 동작할 내용을 코딩
    }
});
```



2. 기본 위젯 다루기 ▶ 버튼과 에디트텍스트

❖ 에디트텍스트 계층도

```
java.lang.Object
```

```
└ android.view.View
```

```
└ android.widget.TextView
```

```
└ android.widget.EditText
```

EditText 계층도

❖ 에디트텍스트 XML과 Java 코드 작성 3단계

```
<EditText
```

```
android:id="@+id/edittext1" />
```

① 에디트텍스트 변수 선언

```
EditText myEdit;
```

② 변수에 에디트텍스트 위젯 대입

```
myEdit = (EditText) findViewById(R.id.edittext1);
```

③ 에디트텍스트에 입력된 값을 가져오기

```
String myStr = myEdit.getText( ).toString( );
```



2. 기본 위젯 다루기 ▶ 버튼과 에디트텍스트

■ hint, textColorHint

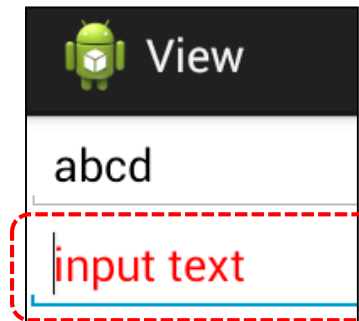
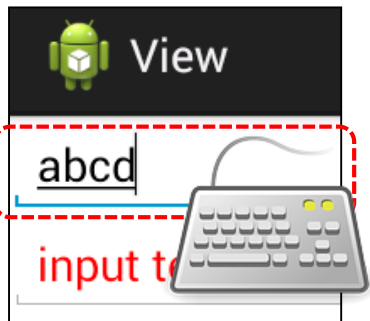
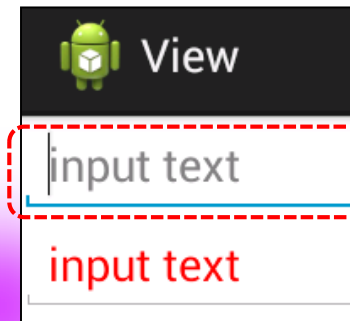
res/layout/edittext_hint_and_text_color_hint.xml

```
<LinearLayout xmlns:android="..."
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:orientation="vertical">

    <EditText android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:hint="input text"/>

    <EditText android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:hint="input text"
        android:textColorHint="#F00"/>

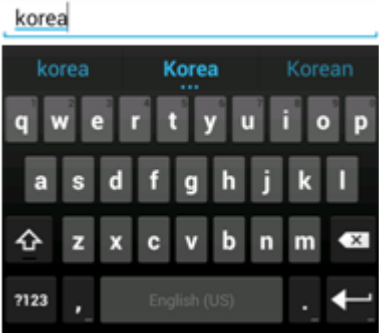
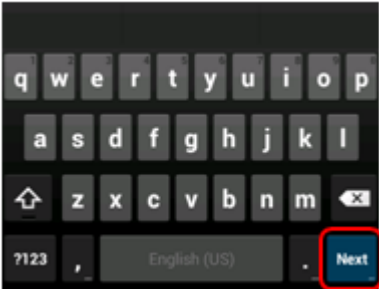
</LinearLayout>
```





2. 기본 위젯 다루기 ▶ 버튼과 에디트텍스트

EditText 속성 - inputType

속성값	의미	적용 예
none	일반적인 입력기 모양	
text	일반적인 입력기 모양에서 Next라는 기능이 추가된다. Next 키를 누르면 현재 에디트텍스트에 있는 커서가 다음 에디트텍스트로 이동한다.	
phone	전화번호를 입력하기 편리한 입력기다.	



2. 기본 위젯 다루기 ▶ 버튼과 에디트텍스트

EditText 속성 - inputType

textNoSuggestions	특정 단어를 한 자씩 입력할 때마다 추천 단어가 상단에 표시된다. 하지만 textNoSuggestions 속성값으로 설정하면 추천어를 표시하지 않는다.	
number	대부분 숫자 키로만 형성된 입력기가 나타난다. 해당 입력기는 양수만 입력 가능하다(소수점, 음수 입력 불가). 숫자 입력 시에는 추천어가 필요치 않으므로 생략된다.	



2. 기본 위젯 다루기 ▶ 버튼과 에디트텍스트

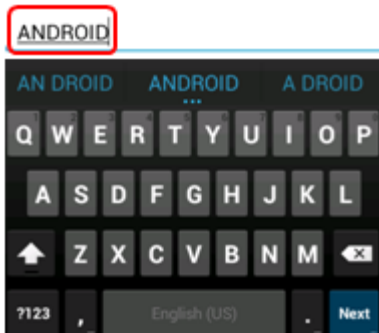
EditText 속성 - inputType

numbersigned	대부분 숫자 키로만 형성된 입력기가 나타난다. 양수와 음수 입력이 가능하다(소수점 입력 불가). 숫자 입력 시에는 추천어가 필요 없으므로 추천어는 제공되지 않는다.	
numberDecimal	대부분 숫자 키로만 형성된 입력기가 나타난다. 해당 입력기는 소수점 입력이 가능하다(음수 입력 불가). 숫자 입력 시에는 추천어가 필요 없으므로 추천어는 제공되지 않는다.	
time	시간 정보를 입력하기에 적합한 입력기가 나타난다. 특히 시분초 사이에 입력하는 기호 : 키가 존재한다. 시간 입력 시에는 추천어가 필요 없으므로 추천어는 제공되지 않는다.	



2. 기본 위젯 다루기 ▶ 버튼과 에디트텍스트

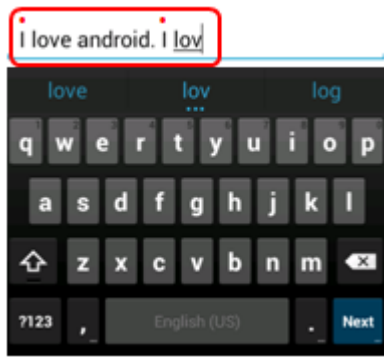
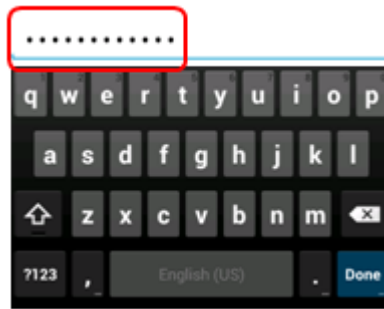
EditText 속성 - inputType

date	날짜 정보를 입력하기에 적합한 입력기가 나타난다. 특히 연월일 사이에 입력하는 기호 / 키가 존재한다. 날짜 입력 시에는 추천어가 필요 없으므로 추천어는 제공되지 않는다.	
datetime	날짜와 시간 정보를 입력하기에 적합한 입력기가 나타난다. 특히 연월일과 시분초 사이에 입력하는 기호 / : 키가 존재한다. 날짜와 시간 입력 시에는 추천어가 필요 없으므로 추천어는 제공되지 않는다.	
textCapCharacters	모든 영문이 대문자로 입력된다. 입력 전에 Caps Lock 키를 해제하면 소문자도 입력 가능하나 한 글자를 입력하고 나면 다시 Caps Lock 키가 활성화되어 대문자 입력을 유도한다	



2. 기본 위젯 다루기 ▶ 버튼과 에디트텍스트

EditText 속성 - inputType

textCapWords	영문 한 단어 첫 자를 대문자로 입력한다. 입력 전에 Caps Lock 키를 해제하면 소문자도 입력 가능하나 한 단어를 입력하고 나면 다시 Caps Lock 키가 활성화되어 단어의 첫 글자 입력을 대문자로 입력하도록 유도한다.	
textCapSentences	영문 한 문장의 첫 자를 대문자로 입력한다. 입력 전에 Caps Lock 키를 해제하면 소문자도 입력 가능하나, 한 문장을 입력하고 나면 다시 Caps Lock 키가 활성화되어 문장의 첫 글자 입력을 대문자로 입력하도록 유도한다.	
textPassword	문자 기반의 입력기 형태이며, 입력되는 글자가 모두 • 기호로 표시된다.	

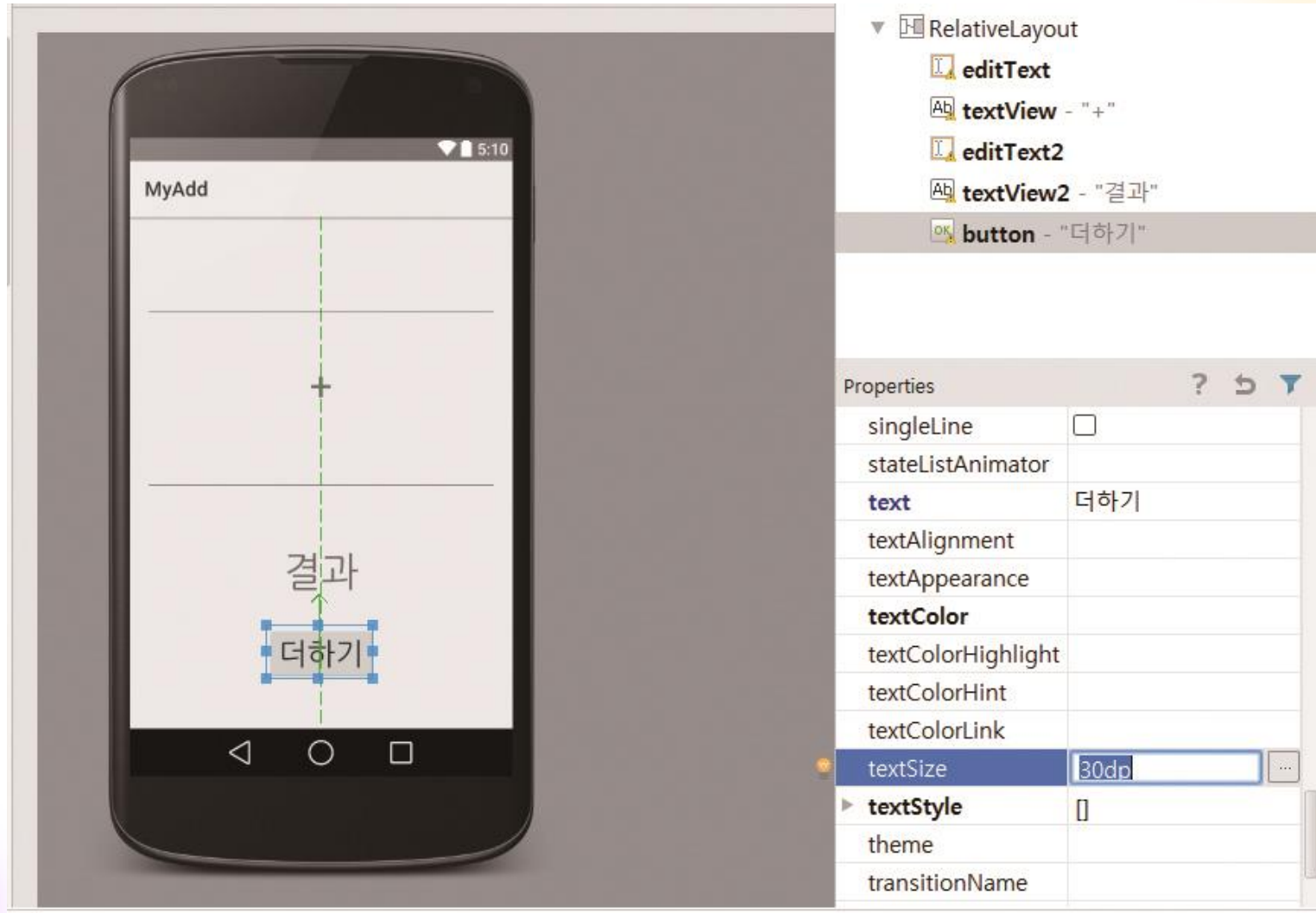


2. 기본 위젯 다루기 ▶ 버튼과 에디트텍스트

numberPassword	숫자 기반의 입력기 형태이며, 입력되는 글자가 모두 • 기호로 표시된다.	 A numeric keypad with digits 1-9, 0, and a Done button. The digits are arranged in a 3x3 grid with 0 and Done at the bottom.
textEmailAddress	이메일 주소를 입력하기 위한 입력기가 나타난다. 특히 메일 주소에 꼭 들어가는 @와 . 기호 키가 추가된다. 이메일 주소 입력 시에는 추천어가 필요치 않으므로 생략된다.	 A QWERTY keyboard with additional keys for @ and . (Next) highlighted with red boxes.
textShortMessage	SMS 메시지를 입력할 때 많이 사용되는 이모티콘 모음 키가 추가된다.	 Two images of a QWERTY keyboard. The top image shows the standard keyboard with the emoji key (:) highlighted with a red box. A blue arrow points down to the bottom image, which shows the keyboard with the emoji key expanded to show a selection of emojis, also highlighted with a red box.



더하기 기능을 위한 MyAdd 안드로이드 프로젝트 만들기





자동으로 붙여지는 ID 생성 규칙

```
<RelativeLayout>  
  <EditText  
    android:id="@+id/editText1" />  
  <TextView  
    android:id="@+id/textView1" />  
  <EditText  
    android:id="@+id/editText2" />  
  <TextView  
    android:id="@+id/textView2" />  
  <Button  
    android:id="@+id/button1" />  
</RelativeLayout>
```



변수 선언과 import 그리고 코드 입력하기

```
public class MainActivity extends Activity {  
    EditText editText1;  
    EditText editText2;  
    TextView textView2;
```

```
protected void onCreate(Bundle savedInstanceState) {  
    super.onCreate(savedInstanceState);  
    setContentView(R.layout.activity_main);  
  
    editText1 = (EditText) findViewById(R.id.editText1);  
    editText2 = (EditText) findViewById(R.id.editText2);  
    textView2 = (TextView) findViewById(R.id.textView2);
```



버튼에 On Click 속성에 값 입력하기

▼ RelativeLayout

- editText
- textView - "+"
- editText2
- textView2 - "결과"
- button - "더하기"**

Properties

maxLength	
maxLines	
maxWidth	
minHeight	
minLines	
minWidth	
nestedScrollingEnabled	☐
onClick	onButton1Clicked
outlineProvider	
padding	
paddingEnd	
paddingStart	
shadowColor	



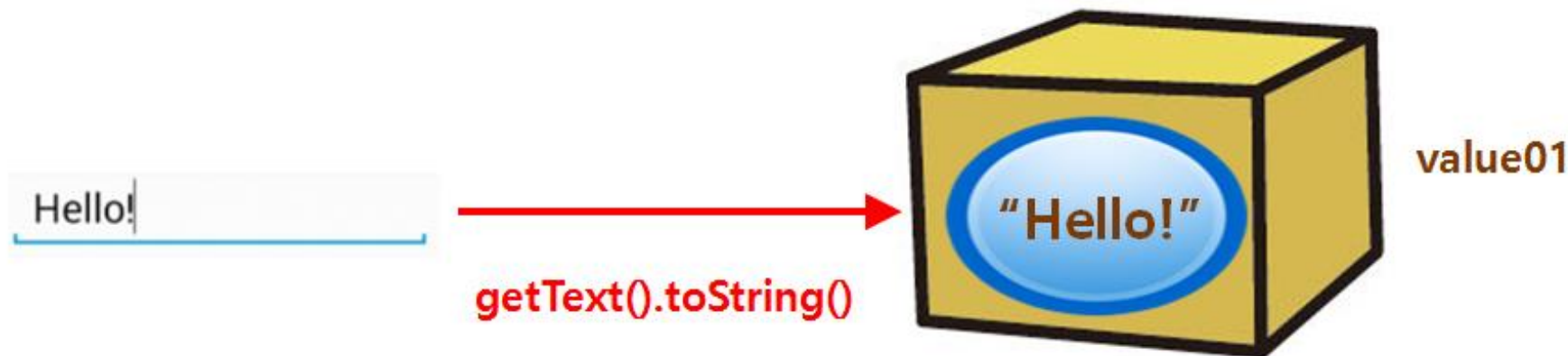
onButton1Clicked 메소드 추가

```
public void onButton1Clicked(View v) {  
  
}
```

```
public void onButton1Clicked(View v) {  
    String value01 = editText1.getText().toString();  
}
```




입력상자에 입력된 값을 코드에서 참조하는 방법



```
public void onButton1Clicked(View v) {  
    String value01 = editText1.getText().toString();  
    String value02 = editText2.getText().toString();  
}
```




값 설정하기와 더한 후 값 설정하기

```
public void onButton1Clicked(View v) {  
    String value01 = editText1.getText().toString();  
    String value02 = editText2.getText().toString();  
    String value03 = "10";  
    textView2.setText(value03);  
}
```

```
public void onButton1Clicked(View v) {  
    String value01 = editText1.getText().toString();  
    String value02 = editText2.getText().toString();  
    int input01 = Integer.parseInt(value01);  
    int input02 = Integer.parseInt(value02);  
  
    int value03 = input01 + input02;  
    textView2.setText(String.valueOf(value03));  
}
```





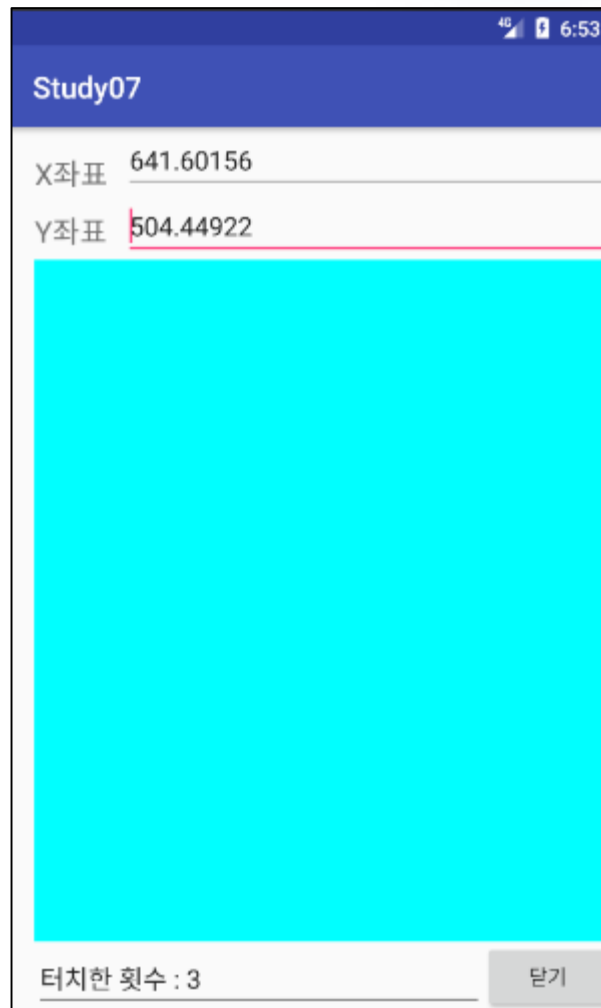
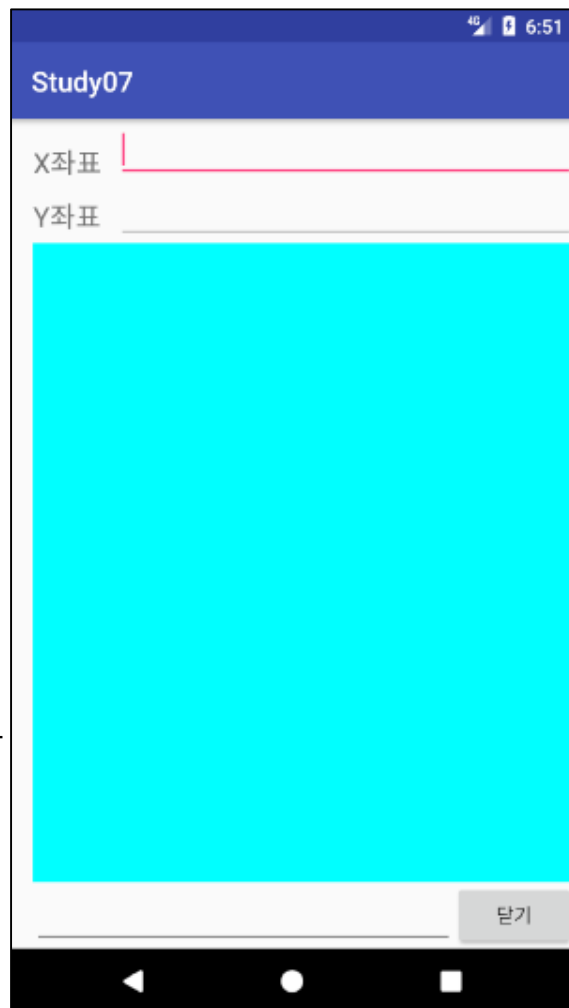
Self-Check-1

1. activity_main.xml 파일을 열고 화면에 입력상자 3개와 버튼 2개를 추가하고 첫번째 버튼에는 '저장', 두번째 버튼에는 '닫기'로 텍스트를 표시합니다
2. 입력상자 3개에는 각각 이름, 나이, 주소를 입력할 수 있도록 합니다
3. MainActivity.java 파일을 열고 소스를 수정합니다.
4. 앱을 실행하고 세 개의 입력 상자에 각각 '김현수', '20', '서울시'라는 토스트 메시지가 보이도록 코드를 입력합니다
5. [닫기]버튼을 눌렀을 때는 앱을 종료하도록 코드를 입력합니다.



Self-Check-2

1. activity_main.xml 파일을 열고 화면에
FrameLayout 한 개를 추가하여 화면의 대부분을
채우고 배경색을 하늘색으로 변경한다
2. 화면 위쪽에 텍스트 뷰 2개와 입력상자 2개,
아래쪽에 입력상자 1개, 버튼 1개를 가로 방향으로
추가합니다. 버튼에는 '닫기'로 텍스트를 표시합니다.
3. MainActivity.java 파일을 열고 소스를 수정합니다.
4. 앱을 실행하고 하늘색 뷰를 터치하면 터치할 때마다
위쪽 입력상자에는 X,Y 좌표값이, 아래쪽 입력상자에는
터치 횟수를 보여주도록 코드를 입력합니다.
5. [닫기]버튼을 눌렀을 때는 앱을 종료하도록 코드를
입력합니다.





2. 기본 위젯 다루기

실습 4-1 초간단 계산기 앱 만들기

❖ 안드로이드 프로젝트 생성

- ✓ 프로젝트 이름 : Project4_1
- ✓ 패키지 이름 : com.cookandroid.project4_1

❖ 화면 디자인 및 편집

- ✓ EditText 2개, Button 4개, TextView 1개 생성
- ✓ 각 위젯에 layout_margin을 적절히 지정
- ✓ TextView는 색상을 빨간색, 글자 크기를 30에
- ✓ 각 위젯의 id는 위에서부터 차례로 Edit1, Edit2, BtnAdd, BtnSub, BtnMul, BtnDiv, TextResult

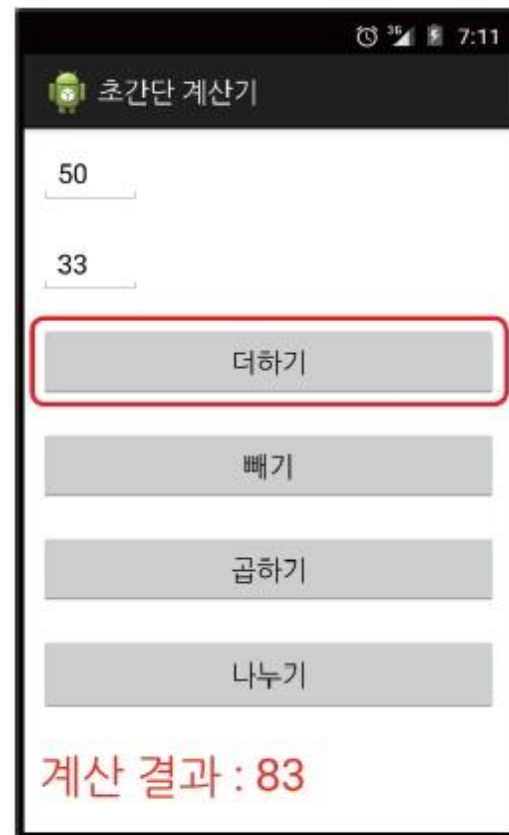


그림 4-5 초간단 계산기 결과 화면



2. 기본 위젯 다루기



실습 4-1 초간단 계산기 앱 만들기

❖ 화면 디자인 : EditText 2개, Button 4개, TextView 1개

[예제 4-19] main.xml

```

1 <LinearLayout>
2   <EditText
3       android:id="@+id/Edit1"
4       android:layout_width="wrap_content"
5       android:layout_height="wrap_content"
6       android:layout_margin="10dp"
7       android:hint="숫자1" />
8   ~~~~ 중간 생략 (에디트텍스트 1개) ~~~~
9   <Button
10      android:id="@+id/BtnAdd"
11      android:layout_width="match_parent"
12      android:layout_height="wrap_content"
13      android:layout_margin="10dp"
14      android:text="더하기" />
15   ~~~~ 중간 생략 (버튼 3개) ~~~~

```

숫자1

숫자2

더하기

빼기

곱하기

나누기

계산 결과 :

```

16 <TextView
17     android:id="@+id/TextResult"
18     android:layout_width="wrap_content"
19     android:layout_height="wrap_content"
20     android:textSize="30dp"
21     android:textColor="#FF0000"
22     android:layout_margin="10dp"
23     android:text="계산 결과 : " />
24 </LinearLayout>

```



2. 기본 위젯 다루기



실습 4-1 초간단 계산기 앱 만들기

❖ Java 코드 작성 및 수정

- ✓ activity_main.xml의 7개 위젯에 대응할 위젯 변수 7개
- ✓ 입력될 2개 문자열을 저장할 문자열 변수 2개
- ✓ 계산 결과를 저장할 정수 변수 1개

예제 4-20 Java 코드 1

```
1  ~~~~ 중간생략 ~~~~
2  public class MainActivity extends Activity {
3      EditText edit1, edit2;
4      Button btnAdd, btnSub, btnMul, btnDiv;
5      TextView textResult;
6      String num1, num2;
7      Integer result;
8
9      @Override
10     public void onCreate(Bundle savedInstanceState) {
11         ~~~~ 중간생략 ~~~~
```




2. 기본 위젯 다루기



실습 4-1 초간단 계산기 앱 만들기

❖ Java 코드 작성 및 수정

- ✓ 버튼에 터치 이벤트 리스너 정의
- ✓ 터치 시에 동작하는 내용을 onTouch() 메소드 안에 코딩

예제 4-22 Java 코드 3

```
1 btnAdd.setOnTouchListener(new View.OnTouchListener() {  
2     public boolean onTouch(View arg0, MotionEvent arg1) {  
3         num1 = edit1.getText().toString();  
4         num2 = edit2.getText().toString();  
5         result = Integer.parseInt(num1) + Integer.parseInt(num2);  
6         textResult.setText("계산 결과 : " + result.toString());  
7         return false;  
8     }  
9 });
```



2. 기본 위젯 다루기

✓ 빼기, 곱하기, 나누기 코드 추가 후 완성

예제 4-23 완성된 Java 코드

```
1 public void onCreate(Bundle savedInstanceState) {
2     super.onCreate(savedInstanceState);
3     setContentView(R.layout.activity_main);
4     setTitle("초간단 계산기");
5
6     edit1 = (EditText) findViewById(R.id.Edit1);
7     edit2 = (EditText) findViewById(R.id.Edit2);
8
9     btnAdd = (Button) findViewById(R.id.BtnAdd);
10    btnSub = (Button) findViewById(R.id.BtnSub);
11    btnMul = (Button) findViewById(R.id.BtnMul);
12    btnDiv = (Button) findViewById(R.id.BtnDiv);
13
14    textResult = (TextView) findViewById(R.id.TextResult);
15
16    btnAdd.setOnTouchListener(new View.OnTouchListener() {
17        public boolean onTouch(View arg0, MotionEvent arg1) {
18            num1 = edit1.getText().toString();
19            num2 = edit2.getText().toString();
20            result = Integer.parseInt(num1) + Integer.parseInt(num2);
21            textResult.setText("계산 결과 : " + result.toString());
22            return false;
23        }
24    });
25}
```




2. 기본 위젯 다루기

```
26 btnSub.setOnTouchListener(new View.OnTouchListener() {
27     public boolean onTouch(View arg0, MotionEvent arg1) {
28         num1 = edit1.getText().toString();
29         num2 = edit2.getText().toString();
30         result = Integer.parseInt(num1) - Integer.parseInt(num2);
31         textResult.setText("계산 결과 : " + result.toString());
32         return false;
33     }
34 });
35
36 btnMul.setOnTouchListener(new View.OnTouchListener() {
37     public boolean onTouch(View arg0, MotionEvent arg1) {
38         num1 = edit1.getText().toString();
39         num2 = edit2.getText().toString();
40         result = Integer.parseInt(num1) * Integer.parseInt(num2);
41         textResult.setText("계산 결과 : " + result.toString());
42         return false;
43     }
44 });
45
46 btnDiv.setOnTouchListener(new View.OnTouchListener() {
47     public boolean onTouch(View arg0, MotionEvent arg1) {
48         num1 = edit1.getText().toString();
49         num2 = edit2.getText().toString();
50         result = Integer.parseInt(num1) / Integer.parseInt(num2);
51         textResult.setText("계산 결과 : " + result.toString());
52         return false;
53     }
54 });
55 }
```



2. 기본 위젯 다루기



[직접 풀어보기 4-3]

❖ <실습 4-1>을 다음과 같이 기능을 추가 또는 변경해 보자.

- ✓ 터치(Touch)가 아닌 클릭(Click)으로 변경
- ✓ 나머지 값 구하기 버튼 추가
- ✓ 값을 입력하지 않고, 버튼을 클릭할 때 오류메시지를 Toast로 나타내기
- ✓ 실수 값 계산하기
- ✓ 0으로 나누면, Toast 메시지를 나타내고 계산하지 않기.

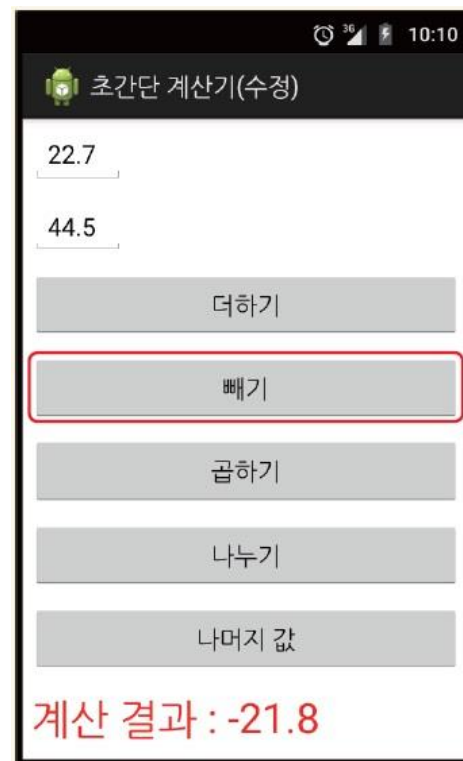


그림 4-6 개선된 계산기



3. 기본 위젯 활용하기 ▶ 컴파운드버튼

❖ 컴파운드버튼 계층도

java.lang.Object

└ android.view.View

└ android.widget.TextView

└ android.widget.Button

└ android.widget.CompoundButton

└ android.widget.CheckBox

└ android.widget.RadioButton

└ android.widget.Switch

└ android.widget.ToggleButton

CompoundButton 계층도

❖ 체크박스 XML과 Java 코드 작성 3단계

[예제 4-24] CheckBox의 XML 코드

```
1 <CheckBox
2     android:id="@+id/android"
3     android:text="안드로이드폰"
4     android:checked="true"/>
5 <CheckBox
6     android:id="@+id/iphone"
7     android:text="아이폰" />
8 <CheckBox
9     android:id="@+id/window"
10    android:text="윈도폰"
11    android:checked="true" />
```

① 체크박스 변수 선언

```
CheckBox mycheck;
```

② 변수에 체크박스 위젯 대입

```
mycheck = (CheckBox) findViewById(R.id.android);
```

③ 체크박스가 변경될 때 동작하는 클래스 정의

```
mycheck.setOnCheckedChangeListener(new CompoundButton.OnCheckedChangeListener() {
    public void onCheckedChanged(CompoundButton arg0, boolean arg1) {
        // 이 부분에 동작할 내용을 코딩
    }
});
```

☒ 안드로이드폰
☐ 아이폰
☒ 윈도폰



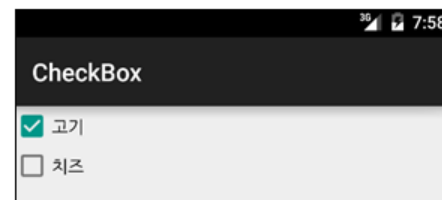
3. 기본 위젯 활용하기 ▶ 컴파운드버튼

- XML로 체크 박스를 선언한다.

main.xml

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:orientation="vertical"
    android:layout_width="fill_parent"
    android:layout_height="fill_parent">
    <CheckBox android:id="@+id/checkbox_meat"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="고기"
        android:onClick="onCheckboxClicked"/>
    <CheckBox android:id="@+id/checkbox_cheese"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="치즈"
        android:onClick="onCheckboxClicked"/>
</LinearLayout>
```

체크박스





3. 기본 위젯 활용하기 ▶ 컴파운드버튼

체크박스의 이벤트 처리

```
public class CheckBoxActivity extends ActionBarActivity {  
    @Override  
    protected void onCreate(Bundle savedInstanceState) {  
        | super.onCreate(savedInstanceState);  
        setContentView(R.layout.main);  
    }  
    public void onCheckboxClicked(View view) {  
        boolean checked = ((CheckBox) view).isChecked();  
  
        switch(view.getId()) {  
            case R.id.checkbox_meat:  
                if (checked)  
                    Toast.makeText(getApplicationContext(), "고기 선택",  
                        Toast.LENGTH_SHORT).show();  
                else  
                    Toast.makeText(getApplicationContext(), "고기 선택 해제",  
                        Toast.LENGTH_SHORT).show();  
                break;  
            case R.id.checkbox_cheese:  
                if (checked)  
                    Toast.makeText(getApplicationContext(), "치즈 선택",  
                        Toast.LENGTH_SHORT).show();  
                else  
                    Toast.makeText(getApplicationContext(), "치즈 선택 해제",  
                        Toast.LENGTH_SHORT).show();  
                break;  
        }  
    }  
}
```



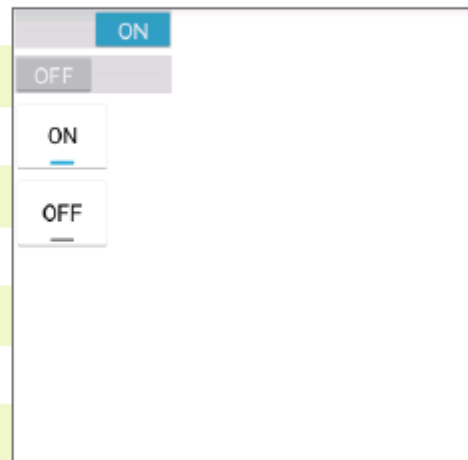
3. 기본 위젯 활용하기 ▶ 컴파운드버튼

❖ 스위치와 토글버튼

- ✓ 스위치와 토글버튼은 모양만 조금 다를 뿐 용도는 거의 동일

예제 4-25 Switch와 ToggleButton의 XML 코드

```
1 <Switch
2     android:checked="true" />
3 <Switch
4     android:checked="false" />
5 <ToggleButton
6     android:checked="true" />
7 <ToggleButton
8     android:checked="false" />
```





3. 기본 위젯 활용하기 ▶ 컴파운드버튼

❖ 라디오버튼과 라디오키트

- ✓ 라디오버튼 : 여러 개 중 하나만 선택해야 하는 경우에 사용
- ✓ 라디오키트 : 라디오버튼만 여러 개 나열하면 클릭하는 것마다 모두 중복 선택이 되므로 라디오키트와 함께 사용해야 함

예제 4-26 RadioGroup과 RadioButton의 XML 코드

```
1 <RadioGroup
2     android:id="@+id/rGroup1" >
3     <RadioButton
4         android:text="남성" />
5     <RadioButton
6         android:text="여성" />
7 </RadioGroup>
```

☐ 남성
☒ 여성



3. 기본 위젯 활용하기 ▶ 라디오버튼

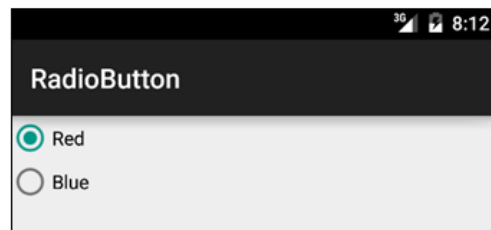
- XML로 라디오 버튼을 정의한다.

main.xml

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical" >

    <RadioGroup
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:orientation="vertical">
        <RadioButton
            android:id="@+id/radio_red"
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:onClick="onRadioButtonClicked"
            android:text="Red"/>
        <RadioButton
            android:id="@+id/radio_blue"
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:onClick="onRadioButtonClicked"
            android:text="Blue"/>
    </RadioGroup>
</LinearLayout>
```

라디오 그룹 안에 라디오 버튼을 2개 정의한다.





3. 기본 위젯 활용하기 ▶ 라디오버튼

라디오 버튼의 이벤트 처리

```
public class RadioButtonActivity extends ActionBarActivity {  
    @Override  
    protected void onCreate(Bundle savedInstanceState) {  
        super.onCreate(savedInstanceState);  
        setContentView(R.layout.main);  
    }  
  
    public void onRadioButtonClicked(View view) {  
        boolean checked = ((RadioButton) view).isChecked();  
  
        switch(view.getId()) {  
            case R.id.radio_red:  
                if (checked)  
                    Toast.makeText(getApplicationContext(),  
                        ((RadioButton) view).getText(),  
                        Toast.LENGTH_SHORT).show();  
                break;  
            case R.id.radio_blue:  
                if (checked)  
                    Toast.makeText(getApplicationContext(),  
                        ((RadioButton) view).getText(),  
                        Toast.LENGTH_SHORT).show();  
                break;  
        }  
    }  
}
```

라디오 버튼의 이벤트를 처리한다.

버튼의 체크 여부를 얻는다.



3. 기본 위젯 활용하기 ▶ 토글버튼

- XML로 라디오 버튼을 정의한다.

main.xml

```
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent" >
```

```
    <ToggleButton
        android:id="@+id/togglebutton"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:textOn="Vibrate on"
        android:textOff="Vibrate off"
        android:onClick="onToggleClicked"
    />
```

```
</LinearLayout>
```



토글 버튼



3. 기본 위젯 활용하기 ▶ 토글버튼

ToggleButtonActivity.java

```
package kr.co.company.togglebutton;
```

```
// 소스만 입력하고 Alt+Enter를 눌러서 import 문장을 자동으로 생성한다.
```

```
public class ToggleButtonActivity extends ActionBarActivity {
```

```
    @Override
```

```
    protected void onCreate(Bundle savedInstanceState) {
```

```
        super.onCreate(savedInstanceState);
```

```
        setContentView(R.layout.main);
```

```
    }
```

```
    public void onToggleClicked(View view) {
```

```
        // Is the toggle on?
```

```
        boolean on = ((ToggleButton) view).isChecked();
```

```
        if (on) {
```

```
            Toast.makeText(getApplicationContext(), "Checked",  
                        Toast.LENGTH_SHORT).show();
```

```
        } else {
```

```
            Toast.makeText(getApplicationContext(), "Not checked",  
                        Toast.LENGTH_SHORT).show();
```

```
        }
```

```
    }
```

```
}
```



3. 기본 위젯 활용하기 ▶ 이미지뷰와 이미지버튼

❖ 이미지뷰(Image View)

- ✓ 그림을 출력하는 위젯으로 그림이 필요하거나 화면을 화려하게 구성할 때 사용
- ✓ 이미지뷰에 보여줄 그림 파일은 프로젝트의 [res]-[drawable]에 있어야 함

```
java.lang.Object
└ android.view.View
    └ android.widget.ImageView
        └ android.widget.ImageButton
```

ImageView 계층도



3. 기본 위젯 활용하기 ▶ 이미지뷰와 이미지버튼

❖ 이미지뷰 및 이미지버튼의 XML 속성

- ✓ **src** : 이미지의 경로를 나타냄
- ✓ **maxHeight/maxWidth** :
이미지의 크기를 지정
- ✓ **scaleType** : 이미지의 확대/축소 방식 지정
- ✓ 그림 파일이 [res]-[drawable] 폴더에 있어야
사용 가능

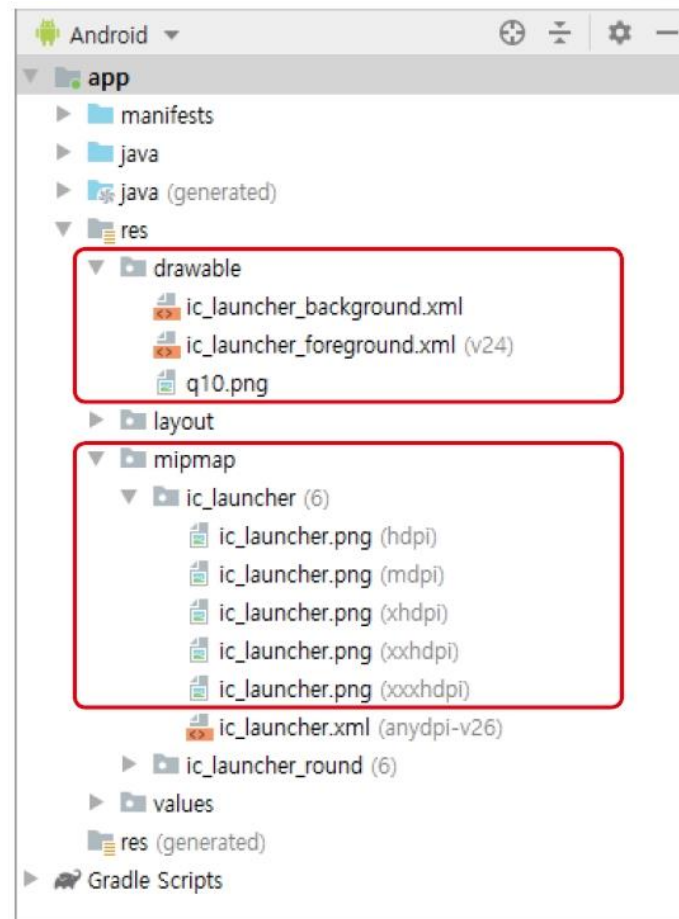


그림 4-7 drawable 폴더와 mipmap 폴더

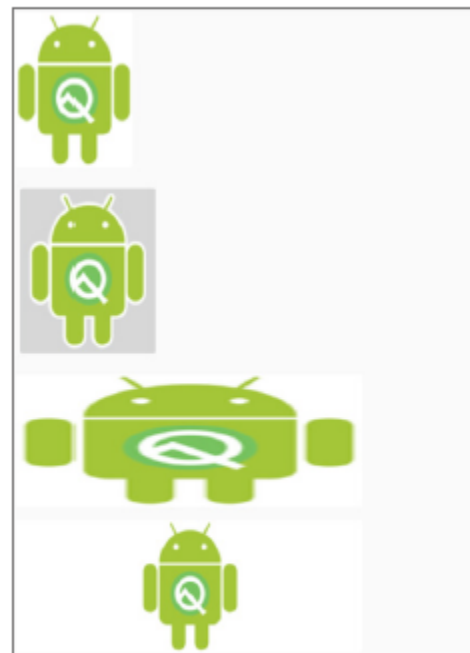


3. 기본 위젯 활용하기 ▶ 이미지뷰와 이미지버튼

❖ 이미지뷰와 이미지버튼의 XML 코드

예제 4-27 ImageView와 ImageButton의 XML 코드

```
1 <ImageView
2     android:src="@drawable/q10.png" />
3 <ImageButton
4     android:src="@drawable/q10.png" />
5 <ImageView
6     android:layout_width="300dp"
7     android:layout_height="100dp"
8     android:scaleType="fitXY"
9     android:src="@drawable/q10.png" />
10 <ImageView
11     android:layout_width="300dp"
12     android:layout_height="100dp"
13     android:scaleType="fitCenter"
14     android:src="@drawable/q10.png" />
```





3. 기본 위젯 활용하기 ▶ 이미지뷰와 이미지버튼

res/layout/imageview_src.xml

```
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="200dp"
    android:layout_height="200dp"
    android:background="#7F7F7F"
    android:orientation="vertical">

    <ImageView android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_marginBottom="10dp"
        android:background="#3F48CC"
        android:src="@drawable/sori_small"/>

    <ImageView android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:background="#3F48CC"
        android:src="@drawable/sori_big"/>

</LinearLayout>
```



원본파일

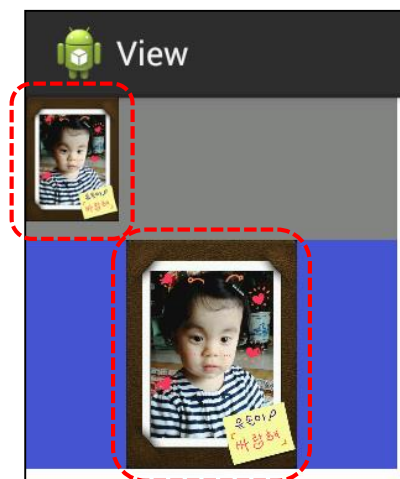
파일명 : sori_big.png
해상도 : 960X1280



원본파일

파일명 : sori_small.png
해상도 : 100X133

이미지 크기에 따라 화면에 출력된 이미지뷰의 모양이 다르다. 왜 그럴까?



3. 기본 위젯 활용하기 ▶ 이미지뷰와 이미지버튼

■ 이미지뷰의 기본 기능을 아는 것이 중요하다.

뷰 영역보다 이미지가 작을 때	뷰 영역보다 이미지가 클 때
	
	

이렇게 원본 이미지를 이미지뷰에 어떻게 보일지를 결정하는 것은 `scaleType` 속성이다. 그리고 속성값 중 기본 값은 `fitCenter`이다.



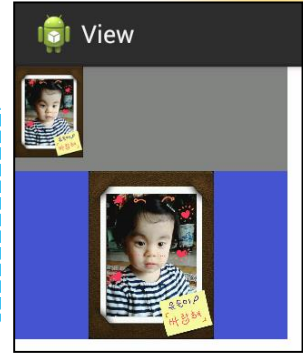
3. 기본 위젯 활용하기 ▶ 이미지뷰와 이미지버튼

■ 그렇다면 예제의 실행 결과를 분석해보자.

원본 이미지 크기



뷰그룹 영역

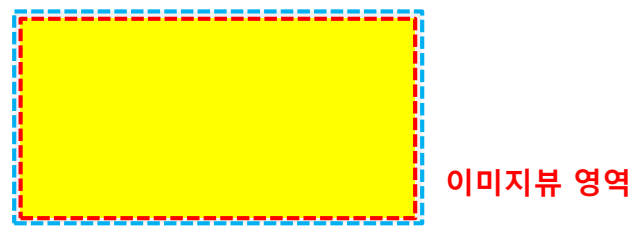


```
<ImageView android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:layout_marginBottom="10dp"
    android:background="#3F48CC"
    android:src="@drawable/sori_small"/>
```

원본 이미지 크기



뷰그룹 영역



```
<ImageView android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:background="#3F48CC"
    android:src="@drawable/sori_big"/>
```



3. 기본 위젯 활용하기 ▶ 이미지뷰와 이미지버튼

ImageView 속성 - adjustViewBounds

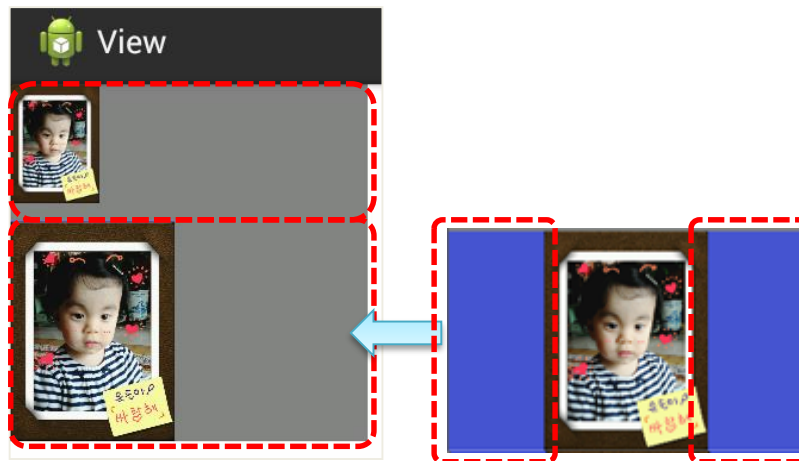
res/layout/imageview_adjust_view_bounds.xml

```
<LinearLayout xmlns:android="..."
    android:layout_width="200dp"
    android:layout_height="200dp"
    android:background="#7F7F7F"
    android:orientation="vertical">

    <ImageView android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_marginBottom="10dp"
        android:background="#3F48CC"
        android:src="@drawable/sori_small"
        android:adjustViewBounds="true"/>

    <ImageView android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:background="#3F48CC"
        android:src="@drawable/sori_big"
        android:adjustViewBounds="true"/>

</LinearLayout>
```



특정 뷰그룹에 이미지뷰를 추가하면, 부모 뷰그룹은 이미지뷰의 영역을 결정해준다.

그리고 이미지뷰는 뷰그룹이 할당한 영역에 자신의 콘텐츠인 원본이미지를 그린다.

여기서 원본이미지가 뷰영역 보다 크다면 종횡비를 유지하여 축소하는데, 대부분 이미지뷰와 원본이미지의 종횡비 차이가 있다. 따라서 이미지뷰에 좌우 빈 공간이 생긴다.

이때 **adjustViewBounds** 속성을 이용하면 이미지뷰 크기를 축소된 원본이미지로 조정한다.

3. 기본 위젯 활용하기 ▶ 이미지뷰와 이미지버튼

ImageView 속성 - maxWidth, maxHeight

이 속성은 이미지뷰 영역에 그려질 **원본 이미지의 최대 크기**를 지정한다.

maxWidth 속성은 최대 너비를 설정하고, maxHeight 속성은 최대 높이를 설정한다.

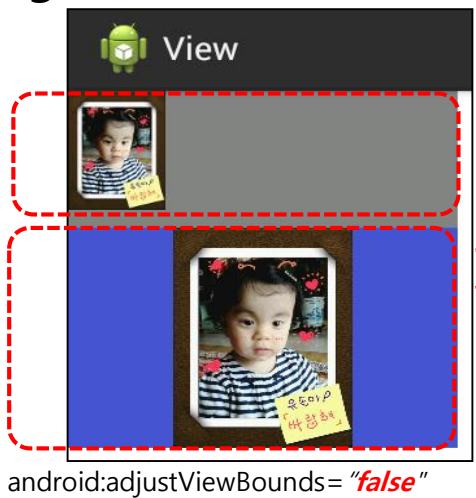
res/layout/imageview_max_width_n_max_height.xml

```
<LinearLayout xmlns:android="..."
    android:layout_width="200dp"
    android:layout_height="200dp"
    android:background="#7F7F7F"
    android:orientation="vertical">

    <ImageView android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_marginBottom="10dp"
        android:background="#3F48CC"
        android:src="@drawable/sori_small"
        android:maxWidth="100dp"
        android:maxHeight="100dp"/>

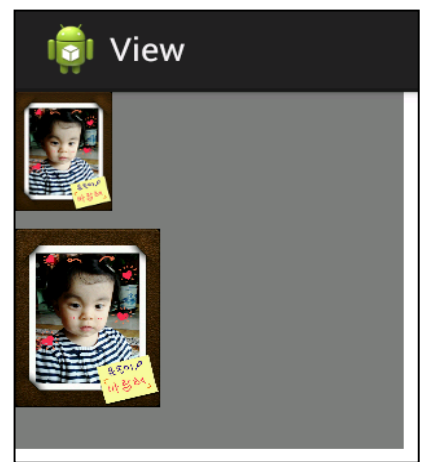
    <ImageView android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:background="#3F48CC"
        android:src="@drawable/sori_big"
        android:adjustViewBounds="false"
        android:maxWidth="100dp"
        android:maxHeight="100dp"/>

</LinearLayout>
```



원본이미지가 100X100보다 작기 때문에 차이가 없다.

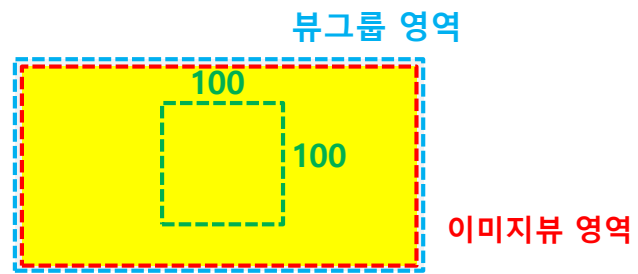
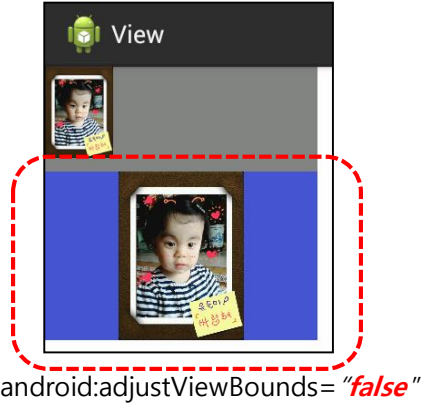
원본 이미지가 100X100보다 훨씬 크지만 왜 차이가 없을까?



android:adjustViewBounds = "true"

3. 기본 위젯 활용하기 ▶ 이미지뷰와 이미지버튼

■ 그렇다면 예제의 실행 결과를 분석해보자.

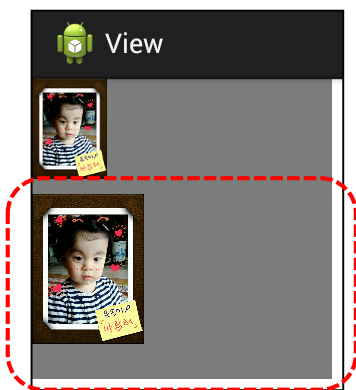


!!! 이미지뷰의 기본 기능은 자신의 영역에
원본이미지를 종횡비를 유지한 채 가득 채운다.
(뒤에서 배울 `scaleType` 속성값 `fitCenter`가 기본)

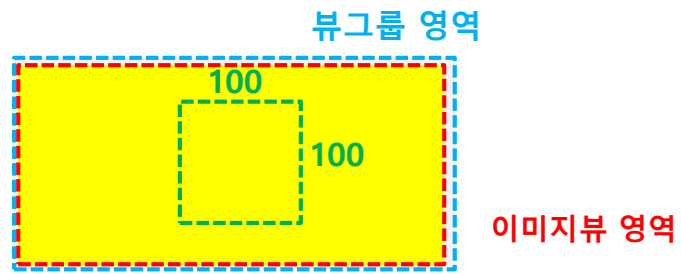
```
<ImageView  
    android:layout_width="wrap_content"  
    android:layout_height="wrap_content"  
    android:layout_marginBottom="10dp"  
    android:background="#3F48CC"  
    android:src="@drawable/sori_small"  
    android:adjustViewBounds="false"  
    android:maxWidth="100dp"  
    android:maxHeight="100dp"/>
```

3. 기본 위젯 활용하기 ▶ 이미지뷰와 이미지버튼

■ 이를 위해 **adjustViewBounds** 설정이 필요하다.



android:adjustViewBounds="true"



```
<ImageView
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:layout_marginBottom="10dp"
    android:background="#3F48CC"
    android:src="@drawable/sori_small"
    android:adjustViewBounds="true"
    android:maxWidth="100dp"
    android:maxHeight="100dp"/>
```



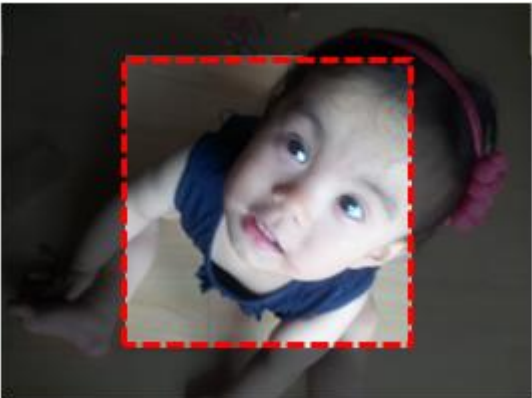
3. 기본 위젯 활용하기 ▶ 이미지뷰와 이미지버튼

이미지뷰에 표시되는 원본이미지 크기를 다양한 형태로 조정할 수 있다.

속성값 및 의미	뷰 영역보다 이미지가 작을 때	뷰 영역보다 이미지가 클 때
matrix 원본이미지를 이미지뷰 좌측 상단부터 표시한다. • 이미지 비율 유지 • 이미지 크기 유지	 빨강색 영역은 이미지 뷰의 영역이다.	 이미지뷰 밖의 영역으로 화면에 보이지 않는 영역
		



3. 기본 위젯 활용하기 ▶ 이미지뷰와 이미지버튼

속성값 및 의미	뷰 영역보다 이미지가 작을 때	뷰 영역보다 이미지가 클 때
center 원본이미지를 이미지뷰 정 중앙에 표시한다. • 이미지 비율 유지 • 이미지 크기 유지		
		

3. 기본 위젯 활용하기 ▶ 이미지뷰와 이미지버튼

속성값 및 의미	뷰 영역보다 이미지가 작을 때	뷰 영역보다 이미지가 클 때
centerInside 원본이미지를 이미지뷰 영역 내에 중앙 정렬하여 맞춘다. 만일 이미지 크기가 뷰 영역보 다 작으면 이미지 크기를 변경 하지 않는다.		
• 이미지 비율 유지 • 뷰 영역보다 이미지가 크면 원본이미지를 뷰 크기에 맞게 변경, 크지 않으면 유지		

3. 기본 위젯 활용하기 ▶ 이미지뷰와 이미지버튼

속성값 및 의미	뷰 영역보다 이미지가 작을 때	뷰 영역보다 이미지가 클 때
centerCrop 원본이미지를 이미지뷰 정 중앙에 표시한다. 원본이미지는 이미지뷰 영역 안에 비율을 유지하면서 여백을 남기지 않고 맞춘다. • 이미지 비율 유지 • 이미지 크기 변경		
		



3. 기본 위젯 활용하기 ▶ 이미지뷰와 이미지버튼

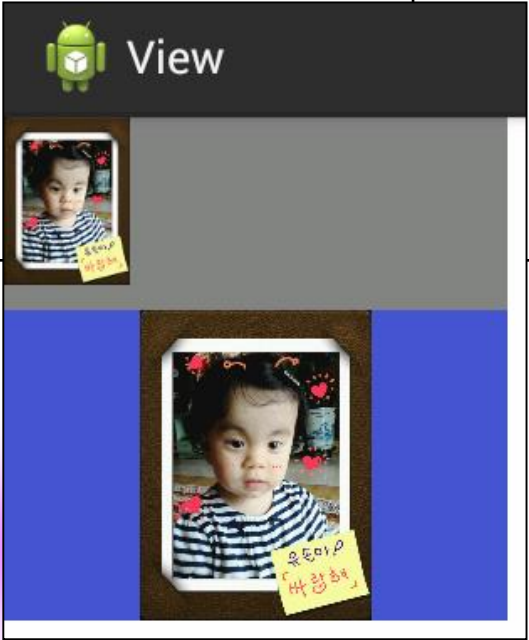
속성값 및 의미	뷰 영역보다 이미지가 작을 때	뷰 영역보다 이미지가 클 때
fitXY 원본이미지를 이미지뷰 영역 안에 여백을 남기지 않고 맞춘다. • 이미지 비율 변경 • 이미지 크기 변경		
		

3. 기본 위젯 활용하기 ▶ 이미지뷰와 이미지버튼

속성값 및 의미	뷰 영역보다 이미지가 작을 때	뷰 영역보다 이미지가 클 때
fitStart 원본이미지를 이미지뷰 영역 안에 좌측 혹은 상단 정렬하여 맞춘다. • 이미지 비율 유지 • 이미지 크기 변경		
		

3. 기본 위젯 활용하기 ▶ 이미지뷰와 이미지버튼

속성값 및 의미	뷰 영역보다 이미지가 작을 때	뷰 영역보다 이미지가 클 때
fitCenter • 이미지 비율 유지 • 이미지 크기 변경		
		



fitCenter 속성은 이미지뷰의 기본 속성 값이다.

3. 기본 위젯 활용하기 ▶ 이미지뷰와 이미지버튼

속성값 및 의미	뷰 영역보다 이미지가 작을 때	뷰 영역보다 이미지가 클 때
fitEnd 원본이미지를 이미지뷰 영역 내에 우측 혹은 하단 정렬하여 맞춘다. • 이미지 비율 유지 • 이미지 크기 변경		
		

■ 속성값들을 쉽게 정리하면 아래와 같다.

- **fit**로 시작하는 속성값은 모두 원본이미지를 이미지뷰 크기만큼 축소하거나 확대한다.
- 이미지 비율을 깨는 속성값은 **fitXY**뿐이다.
- **matrix**와 **center** 속성값은 원본이미지의 크기를 변경하지 않는다.



3. 기본 위젯 활용하기 ▶ 이미지뷰와 이미지버튼

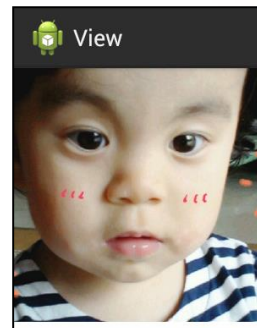
res/layout/imageview_scale_type.xml

```
<LinearLayout xmlns:android="..."
    android:layout_width="200dp"
    android:layout_height="200dp"
    android:orientation="horizontal">

    <ImageView android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:src="@drawable/sori_big"
        android:scaleType="center"/>

</LinearLayout>
```

원본 이미지 크기





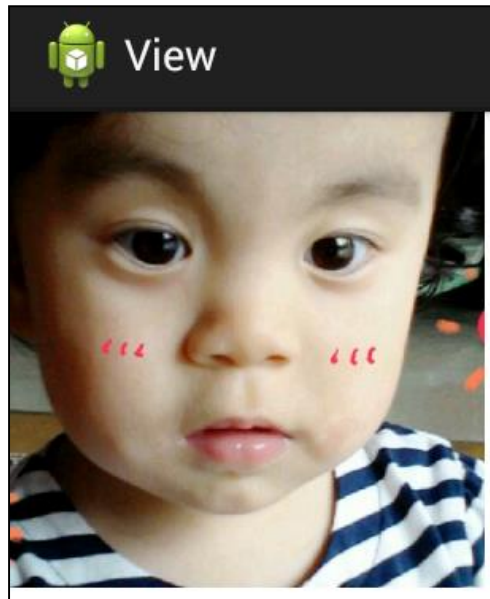
3. 기본 위젯 활용하기 ▶ 이미지뷰와 이미지버튼

res/layout/imageview_crop_to_padding.xml

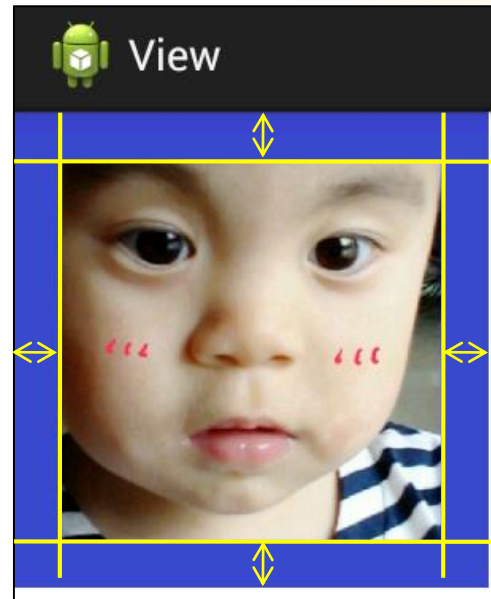
```
<LinearLayout xmlns:android="..."
    android:layout_width="200dp"
    android:layout_height="200dp"
    android:orientation="horizontal">

    <ImageView android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:src="@drawable/sori_big"
        android:background="#3F48CC"
        android:padding="20dp"
        android:scaleType="center"
        android:cropToPadding="false"/>

</LinearLayout>
```



android:cropToPadding = "*false*"



android:cropToPadding = "*true*"



3. 기본 위젯 활용하기



실습 4-2 좋아하는 애완동물 선택 앱 만들기

❖ 안드로이드 프로젝트 생성

- ✓ 프로젝트 이름 : Project4_2
- ✓ 패키지 이름 : com.cookandroid.project4_2





3. 기본 위젯 활용하기



실습 4-2 좋아하는 애완동물 선택 앱 만들기

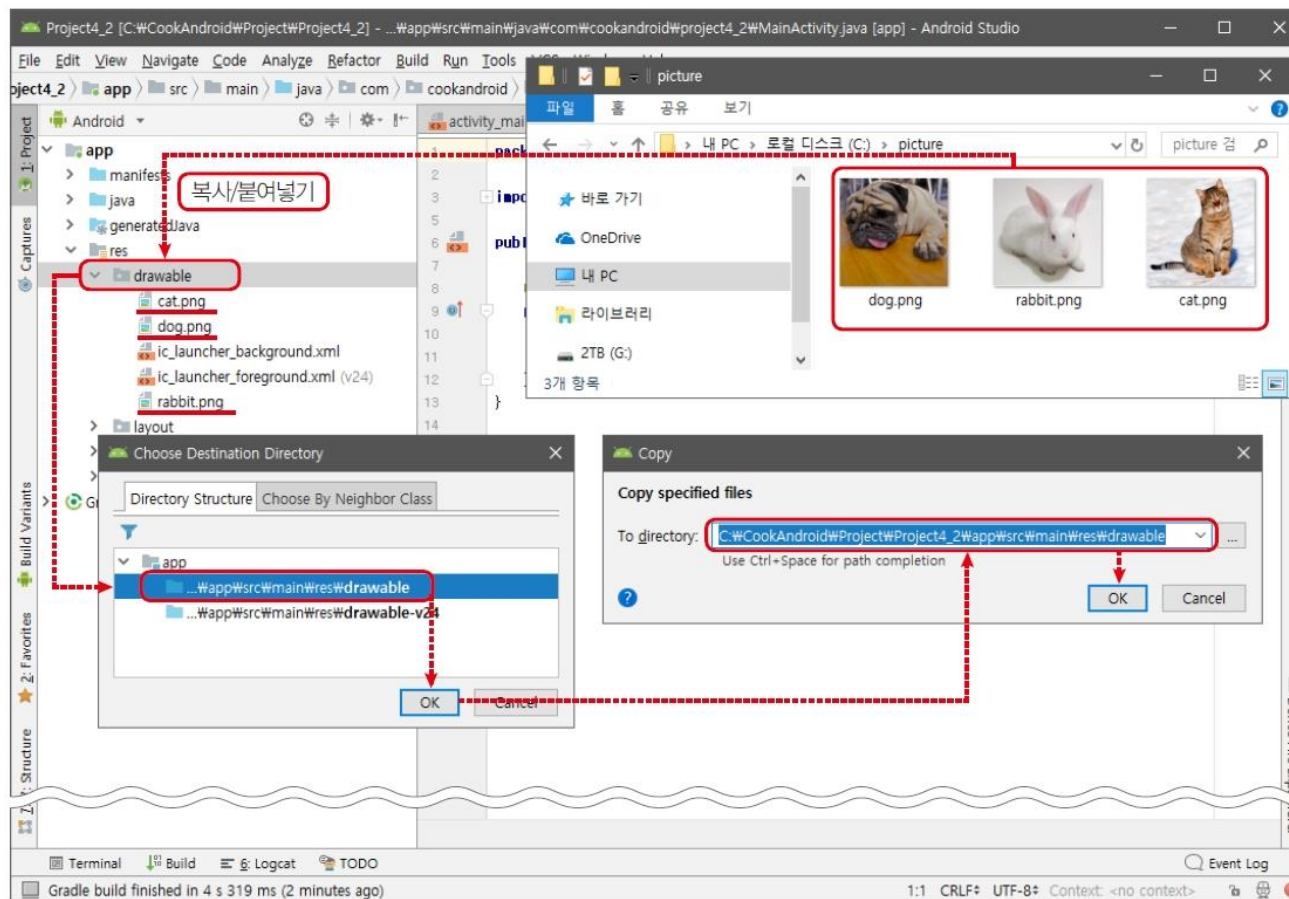


그림 4-9 그림 파일 복사



3. 기본 위젯 활용하기



실습 4-2 좋아하는 애완동물 선택 앱 만들기

❖ 화면 및 디자인 편집

- ✓ TextView, CheckBox, TextView, RadioGroup, RadioButton 각각 세 개, Button, ImageView의 차례로 만들
- ✓ 레이아웃에 padding을 적절히 지정
- ✓ 위의 TextView와 CheckBox를 제외하고, 나머지 위젯은 visibility 속성을 invisible로 지정
- ✓ 위젯의 id는 위에서부터 Text1, ChkAgree, Text2, Rgroup1, RdoDog, RdoCat, RdoRabbit, BtnOK, ImgPet으로 함

예제 4-28 activity_main.xml 코드

```
1 <TextView
2     android:id="@+id/Text1"
3     android:text="선택을 시작하겠습니까?" />
4 <CheckBox
5     android:id="@+id/ChkAgree"
6     android:text="시작함" />
7 <TextView
8     android:text="좋아하는 애완동물은?"
9     android:visibility="invisible" />
10 <RadioGroup
11     android:id="@+id/Rgroup1"
12     android:visibility="invisible" >
13     <RadioButton
14         android:id="@+id/RdoDog"
15         android:text="강아지" />
16     ~~~~ 중간 생략 (라디오버튼 2개) ~~~~
17 </RadioGroup>
18 <Button
19     android:id="@+id/BtnOK"
20     android:text="선택 완료"
21     android:visibility="invisible" />
22 <ImageView
23     android:id="@+id/ImgPet"
24     android:visibility="invisible" />
```

선택을 시작하겠습니까?
☐ 시작함



3. 기본 위젯 활용하기



실습 4-2 좋아하는 애완동물 선택 앱 만들기

❖ Java 코드 작성 및 수정

- ✓ activity_main.xml의 9개 위젯에 대응할 위젯 변수 9개

예제 4-29 Java 코드 1

```
1  ~~~~ 중간 생략 ~~~~
2  public class MainActivity extends Activity {
3      TextView text1, text2;
4      CheckBox chkAgree;
5      RadioGroup rGroup1;
6      RadioButton rdoDog, rdoCat, rdoRabbit;
7      Button btnOK;
8      ImageView imgPet;
9
10     @Override
11     public void onCreate(Bundle savedInstanceState) {
12     ~~~~ 중간 생략 ~~~~
```



3. 기본 위젯 활용하기



실습 4-2 좋아하는 애완동물 선택 앱 만들기

❖ Java 코드 작성 및 수정

✓ 각 위젯을 변수에 대입

예제 4-30 Java 코드 2

```
1 public void onCreate(Bundle savedInstanceState) {  
2     super.onCreate(savedInstanceState);  
3     setContentView(R.layout.activity_main);  
4     setTitle("애완동물 사진 보기");  
5  
6     text1 = (TextView) findViewById(R.id.Text1);  
7     chkAgree = (CheckBox) findViewById(R.id.ChkAgree);  
8  
9     text2 = (TextView) findViewById(R.id.Text2);  
10    rGroup1 = (RadioGroup) findViewById(R.id.Rgroup1);  
11    rdoDog = (RadioButton) findViewById(R.id.RdoDog);  
12    rdoCat = (RadioButton) findViewById(R.id.RdoCat);  
13    rdoRabbit = (RadioButton) findViewById(R.id.RdoRabbit);  
14  
15    btnOK = (Button) findViewById(R.id.BtnOK);  
16    imgPet = (ImageView) findViewById(R.id.ImgPet);  
17 }
```




3. 기본 위젯 활용하기



실습 4-2 좋아하는 애완동물 선택 앱 만들기

❖ Java 코드 작성 및 수정

✓ <시작함> 체크박스를 체크/언체크할 때 동작하는 리스너를 onCreate() 내부에 정의

예제 4-31 Java 코드 3

```
1  chkAgree.setOnCheckedChangeListener(new CompoundButton.  
    OnCheckedChangeListener() {  
2      public void onCheckedChanged(CompoundButton arg0, boolean arg1) {  
3  
4          if (chkAgree.isChecked() == true) {  
5              text2.setVisibility(android.view.View.VISIBLE);  
6              rGroup1.setVisibility(android.view.View.VISIBLE);  
7              btnOK.setVisibility(android.view.View.VISIBLE);  
8              imgPet.setVisibility(android.view.View.VISIBLE);  
9          } else  
10             {  
11                 text2.setVisibility(android.view.View.INVISIBLE);  
12                 rGroup1.setVisibility(android.view.View.INVISIBLE);  
13                 btnOK.setVisibility(android.view.View.INVISIBLE);  
14                 imgPet.setVisibility(android.view.View.INVISIBLE);  
15             }  
16         }  
17     });
```



3. 기본 위젯 활용하기



실습 4-2 좋아하는 애완동물 선택 앱 만들기

❖ Java 코드 작성 및 수정

- ✓ <선택 확인>을 클릭하면 동작하는 리스너를 onCreate() 메소드 내부에 정의

예제 4-32 Java 코드 4

```
1 btnOK.setOnClickListener(new View.OnClickListener () {
2     public void onClick(View arg0) {
3         switch(rGroup1.getCheckedRadioButtonId()) {
4             case R.id.RdoDog:
5                 imgPet.setImageResource(R.drawable.dog);
6                 break;
7             case R.id.RdoCat:
8                 imgPet.setImageResource(R.drawable.cat);
9                 break;
10            case R.id.RdoRabbit:
11                imgPet.setImageResource(R.drawable.rabbit);
12                break;
13            default:
14                Toast.makeText(getApplicationContext(), "동물 먼저 선택하세요", Toast.
15                    LENGTH_SHORT).show();
16            }
17 });
```



3. 기본 위젯 활용하기

▶ 직접 풀어보기 4-4

[실습 4-2]를 다음과 같이 수정하라.

- ‘좋아하는 안드로이드 버전은?’으로 질문을 변경한다.
- ‘시작함’을 Switch로 변경한다.
- <선택 완료>를 없애고, 라디오버튼을 선택할 때마다 즉시 해당 이미지가 나오도록 변경한다.
- 마지막에 <종료>와 <처음부터>를 추가한다. <종료>를 클릭하면 응용 프로그램이 완전히 종료되게 하고, <처음부터>를 클릭하면 다시 초기화되고 처음 화면이 나오게 한다.

그림 4-10 안드로이드 사진 보기

